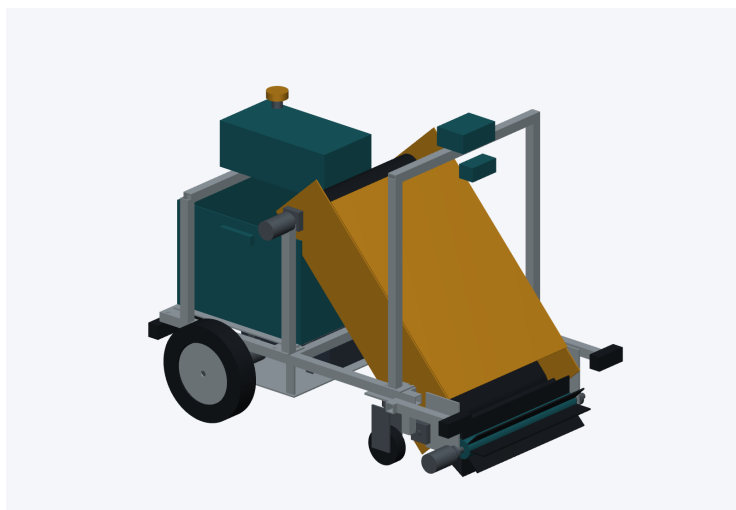


UBOR-JAVA

R02 · Симуляция на WPILib

Книга проекта робота-сборщика мусора



Что нового в этой редакции

Параметрическая модель приводов, виртуальные датчики HAL, телеметрия Field2d и сменный физический backend. Общий Java-контроллер и пульт сохраняются.

Статус выпуска

Интеграция WPILib включена в исходники. Бинарные зависимости в среде подготовки недоступны; компиляция и исполнение WPILib не проверены. Выполненные проверки ядра и интерфейса не заменяют эту проверку.

Далее без изменений приведена книга R01 (37 страниц). Новая глава 25 находится после неё. Прежние результаты относятся к R01; актуальный статус R02 дан в приложении и `verification/REPORT_R02_RU.md`.

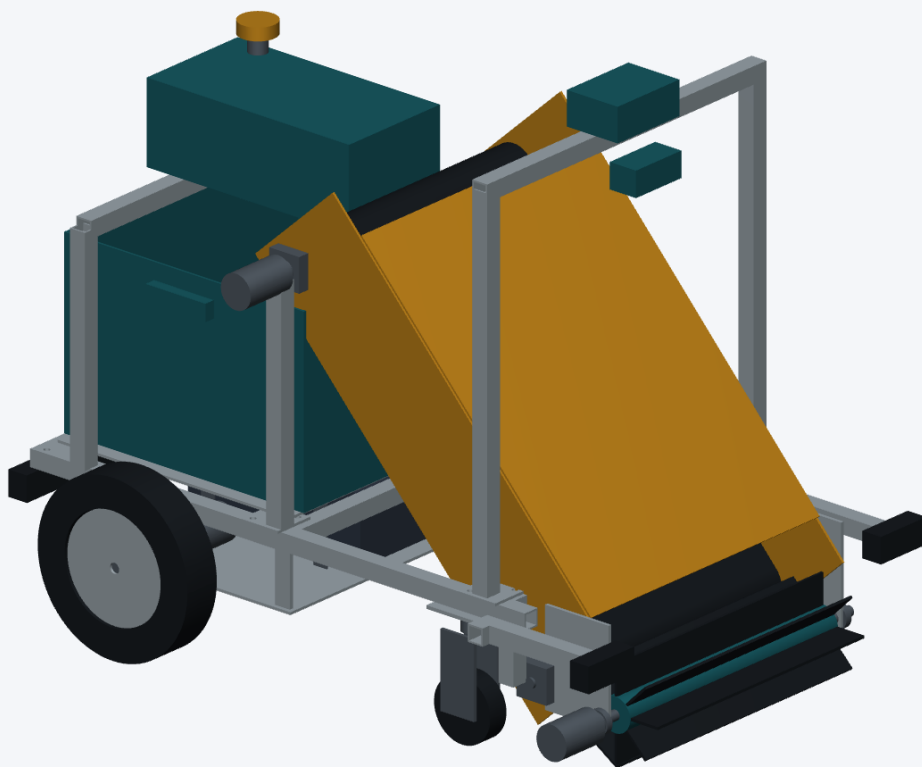
23 сентября 2026 · инженерный прототип, не документация для немедленного изготовления

МЕХАНИКА / ЭЛЕКТРОНИКА / JAVA / ML

UBOR-JAVA

Робот-сборщик мусора

Практическая книга инженерного проекта R01



От параметрической CAD-сборки до пульта и обучения CNN.
Исходники, расчёты, протоколы и честные границы проверки.

23 сентября 2026 · Инженерный proof of concept

Не производственный выпуск. Аппаратные испытания и обучение ещё не выполнены.

Содержание

24 главы, предисловие и три приложения. Номера страниц и ссылки ведут к соответствующим разделам.

Предисловие. От идеи к проверяемой машине	3
01. Какую задачу мы действительно решаем	4
02. Архитектура: четыре независимых вопроса	5
03. Механическая компоновка и единый источник геометрии	7
04. Тяга, скорость, масса и запас энергии	9
05. Щётка, лента и геометрия приёма	10
06. Сборка, обслуживание и размерные цепи	11
07. Электропитание: от батареи до двигателя	13
08. Четыре одинаковых канала управления мотором	14
09. Аппаратный останов, который не зависит от нейросети	15
10. Шины, GPIO и электрические интерфейсы	17
11. Java-ядро: данные, свежесть и состояния	18
12. Запускаем собранный симулятор	19
13. Цикл управления и регуляторы колёс	21
14. Пульт, геймпад и управление по сети	22
15. Pi4J: от Java-объекта к байтам на проводе	23
16. Камера, координаты и границы машинного зрения	24
17. Сбор данных при объезде	25
18. Разбиение, обучение и оценка DL4J	26
19. Перенос ML на Raspberry Pi и проверка задержек	27
20. Что показывают фактические проверки R01	28
21. Стендовый ввод: последовательность разрешений	30
22. Остановка и геометрический запас	31
23. Анализ отказов и незакрытые инженерные задачи	32
24. Воспроизводимость и выпуск следующей ревизии	33
Приложение А. Карта файлов и команды	34
Приложение В. Паспорт проверенной ревизии	36
Приложение С. Первичные источники и проверка предпосылок	37

Предисловие. От идеи к проверяемой машине

Робот, который узнаёт бутылку на фотографии, и робот, который доставляет эту бутылку в бункер, — разные устройства. Между ними лежат геометрия входного окна, сцепление колёс, ток двигателя, задержка камеры, состояние ленты и решение оператора. Эта книга описывает один связанный проект, а не коллекцию независимых примеров. Его имя — UBOR-JAVA R01.

Проект создан для проверки идеи на закрытом полигоне. Он включает настоящую твердотельную CAD-модель, схему соединений электроники, исходный Java-проект, запускаемый симулятор и код обучения. При этом физический робот ещё не построен, аппаратные модули не прошли проверку на Raspberry Pi, а обученных весов в комплекте нет. Эти границы важны: хороший инженерный прототип начинается с возможности точно сказать, что уже проверено, а что пока является гипотезой.

Слово «прошивка» здесь используется в бытовом смысле бортового программного обеспечения. Самостоятельной платы с STM32 и программой на C в R01 нет. На Raspberry Pi работает Java-процесс, который читает датчики, формирует команды приводам, принимает пульт и вызывает нейросети. Таймер PWM, аппаратные счётчики энкодеров и сторожевой таймер выполняются покупными микросхемами без нашей микропрограммы. Linux, JVM и численные библиотеки, разумеется, не переписываются на Java.

В книге последовательно разделены расчёт, программа и эксперимент. Например, расчётная тяга 3,15 Н·м на колесо не является измеренным моментом выбранного мотора; успешный TCP-тест не означает испытанную радиосвязь на улице; три собранных предмета в симуляторе не доказывают качество машинного зрения. Такое разделение позволяет дорабатывать проект без потери доверия к его результатам.

Основной маршрут чтения: сначала область применения и архитектура, затем механика и электрическая часть, после этого Java, обучение и стендовый выпуск. Тем, кто хочет немедленно увидеть работающую программу, достаточно открыть главу 12. Книга сопровождает конкретные файлы R01: названия классов и команд соответствуют исходникам в архиве.

Статус издания: инженерный proof of concept, 23 сентября 2026 года. Не рабочая документация для немедленного изготовления, не руководство по эксплуатации сертифицированного изделия.

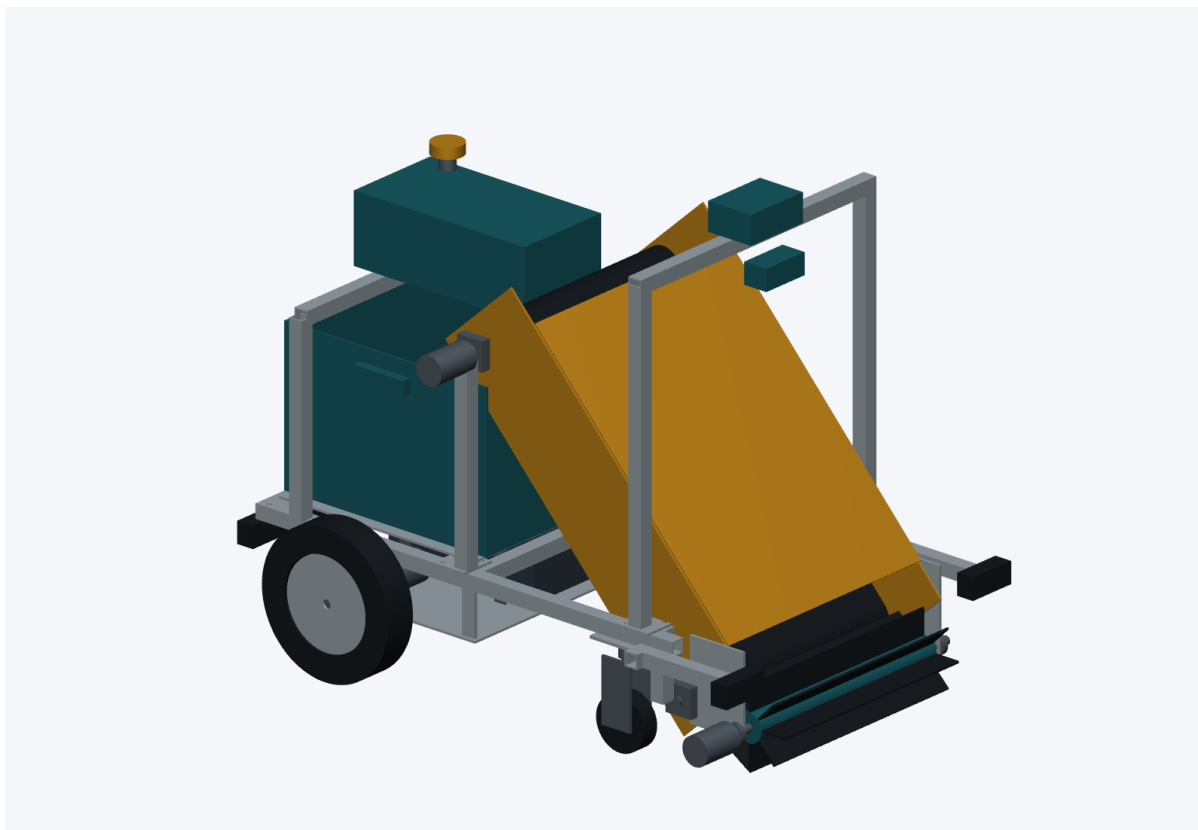


Рисунок 1. Общий вид, программно полученный из той же CAD-сборки, которая передаётся в STEP.

01. Какую задачу мы действительно решаем

Первое ограничение — не вычислительная мощность, а определение мусора. Для R01 допускаются сухая бумага, пустая пластиковая бутылка и пустая металлическая банка. Предмет должен находиться на открытой ровной поверхности, быть доступен переднему подборщику и не содержать опасного вещества. Осколки, шприцы, аккумуляторы, провода, жидкость, горячие предметы и неизвестные упаковки не входят в испытательный набор. Даже пустую банку предварительно осматривает человек.

Классы программы называются NONE, PAPER, PLASTIC_BOTTLE, METAL_CAN, HAZARD и UNKNOWN. Первые три предметных класса допускаются к подбору. NONE обозначает отсутствие цели. UNKNOWN означает недостаточно определённый объект. HAZARD нужен для отказа, но распознавание этого класса не заменяет охрану рабочей зоны. На первых испытаниях вместо опасного предмета используется безопасный макет с соответствующей разметкой.

Робот не сортирует собранный материал по нескольким контейнерам. Здесь один бункер. Классификация отвечает на вопрос «можно ли принять этот предмет», а не управляет распределением фракций. Для настоящей раздельной сортировки потребуются заслонка, несколько накопителей, контроль положения заслонки и новая пространственная модель траектории падения. В R01 этих деталей и этой логики нет.

Рабочая зона — ограждённый полигон без прохожих, лестниц, транспорта, детей и животных. Оператор находится рядом и видит машину непосредственно. Предполагается

ровное сухое покрытие; расчёт уклона 5° служит ориентиром для тяги, а не разрешением работать на любом склоне. объезд препятствий в неизвестной среде, построение карты, движение между кварталами и автоматическая зарядная станция не реализованы.

Параметр	Принято в R01	Что нужно подтвердить
Полная масса	30 кг	Взвешивание собранной машины
Полезная нагрузка	Не более 4 кг	Прочность, торможение, устойчивость
Ручная скорость	До 0,30 м/с	Фактическая скорость под нагрузкой
Автоматическая скорость	До 0,20 м/с	Задержка зрения и остановка
Скорость при подборе	До 0,08 м/с	Геометрия захвата, проскальзывание
Покрытие	Сухое, ровное, закрытая площадка	Испытание колёс и губы подборщика
Сортировка	Только допуск/отказ, один бункер	Дополнительные фракции не поддерживаются

Критерий успеха первого этапа прост: оператор безопасно подводит машину к известному предмету, предмет входит на ленту, задний датчик подтверждает его выход, а после потери команды робот прекращает движение. Только после этого имеет смысл поручать нейросети выбор направления и момента начала подбора. Иначе обучение будет маскировать механическую проблему, которую никакой датасет не исправит.

02. Архитектура: четыре независимых вопроса

В любой момент система должна ответить на четыре вопроса: что перед машиной, что оператор разрешил, в каком состоянии механика и можно ли вообще включать приводы. Эти ответы разделены между восприятием, командным каналом, датчиками и контуром разрешения. Ни один ответ сам по себе не является приказом вращать мотор.

Верхний уровень — камера и две DL4J-модели. Классификатор оценивает фрагменты изображения. Модель движения по полному кадру предлагает линейную и угловую скорость. Предложения поступают в SafetyController, который проверяет свежесть, допустимость объекта, препятствие, пульт и состояние машины. Только после этого DriveControl превращает разрешённую скорость в сигналы левого и правого двигателя.

Нижний программный уровень — PiHardware. Он реализует чтение входов и обмен байтами с периферией через Pi4J. SPI нужен АЦП, расширителю направлений и двум счётчикам энкодеров. I²C нужен многоканальному генератору PWM. Непосредственные GPIO используются для блокировок, heartbeat, программного разрешения и оптических датчиков.

Независимый аппаратный уровень — физический аварийный останов, последовательная цепь нормально замкнутых контактов, TPS3431, триггер ручного повторного пуска и контактор K1. Он не спрашивает нейросеть, безопасно ли продолжать движение. Потеря

heartbeat сбрасывает аппаратное разрешение. Возобновление Java-процесса не должно автоматически вернуть питание приводам.

UBOR-JAVA R01 / потоки управления

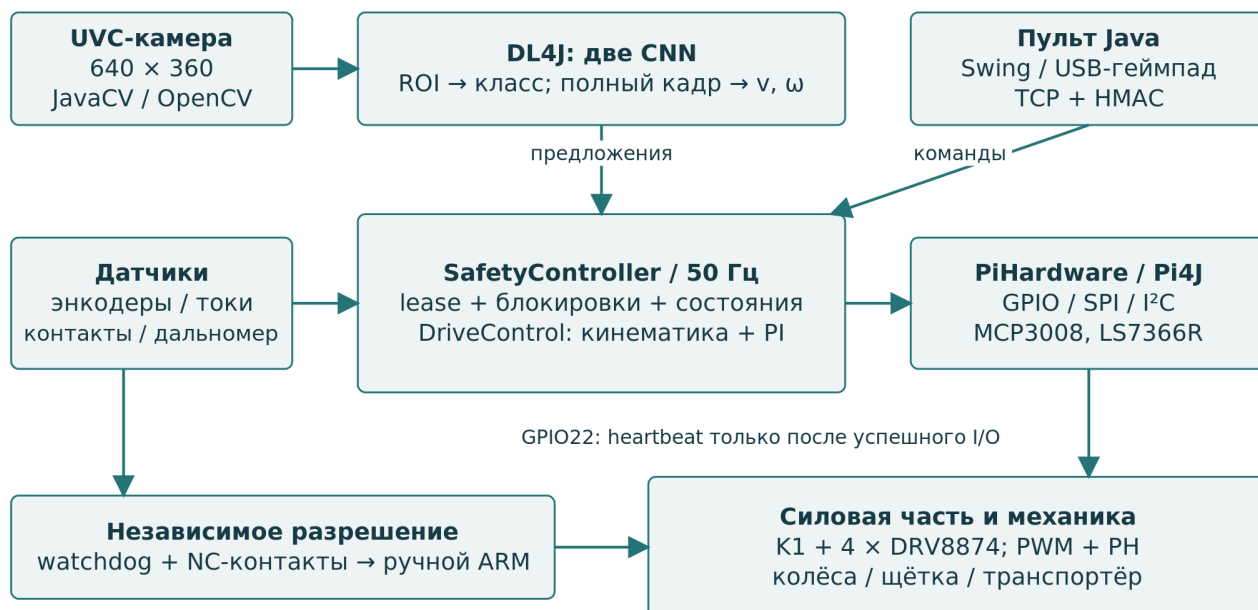


Рисунок 2. Структура R01: нейросеть предлагает, контроллер ограничивает, аппаратный контур отдельно разрешает питание.

Модули Maven повторяют эту границу. `robot-core` не зависит ни от `Pi4J`, ни от `DL4J`; его можно проверять без Интернета. `robot-app` содержит симулятор, пульт, транспорт и исполняемый вход. `robot-pi` связывает байтовые интерфейсы с настоящими шинами. `robot-ml` содержит камеру, обработку изображений, сети и обучение. Такая структура не делает аппаратные ошибки невозможными, но позволяет определить, на какой границе возникло расхождение.

Все собственные алгоритмы управления, обучения и пульта написаны на Java. Python-файлы в архиве — инструменты генерации CAD, электрических исходников и документов. Они не работают на борту вместо заявленного Java-кода. В свою очередь, `DL4J` использует нативный вычислительный backend, а доступ к аппаратуре и видео зависит от Linux и нативных библиотек. Это Java-приложение, а не обещание чистой Java до последнего машинного такта. [1–3]

03. Механическая компоновка и единый источник геометрии

Механическая модель создана в CadQuery/OpenCascade. Исходник `cad/src/build_robot.py` формирует именованные тела и экспортирует сборку, отдельные детали и проекции. Из этой же геометрии получены изображения книги. Здесь нет независимой красивой картинки, по которой затем приблизительно восстановлен чертёж: исходная геометрия одна.

Система координат: X направлен вперёд, Y — поперёк машины, Z — вверх. Плоскость пола имеет $Z = 0$. Ведущая ось расположена около $X = -190$ мм, передние опоры поддерживают зону подбора. Колея ведущих колёс — 500 мм, диаметр — 200 мм. Рама в плане занимает 660×430 мм и выполнена из квадратного профиля $20 \times 20 \times 2$ мм.

Габарит STEP при обратном чтении: $809,5 \times 545,0 \times 665,0$ мм. При вычислении исходной сборки с допусками ядра получены очень близкие значения $809,5 \times 545,14 \times 665,07$ мм. Для компоновки следует пользоваться округлённым размером около $810 \times 545 \times 665$ мм, а при проверке конкретного зазора измерять соответствующие поверхности, не вычитая округлённые габариты.

В сборке 121 именованное твердотельное тело. Число тел не равно числу закупаемых артикулов: отдельные тела могут представлять части условного покупного узла. Упрощённый мотор, подшипниковая опора и корпус камеры показывают занимаемый объём и предполагаемый интерфейс. Они не являются каталоговыми моделями выбранного производителя. До закупки и выпуска рабочих чертежей эти интерфейсы нужно заменить фактическими.

Бункер расположен позади верхнего ролика. Батарея — ниже, на отдельной опоре; электроника — выше бункера на собственной несущей конструкции. Бункер извлекается назад, а не сквозь расположенный сверху блок электроники. Сервисный STEP показывает его смещение на 420 мм назад. Для обслуживания за машиной необходима свободная зона; наличие сервисной конфигурации не доказывает, что провод или забытый болт не перекроет эту зону после сборки.

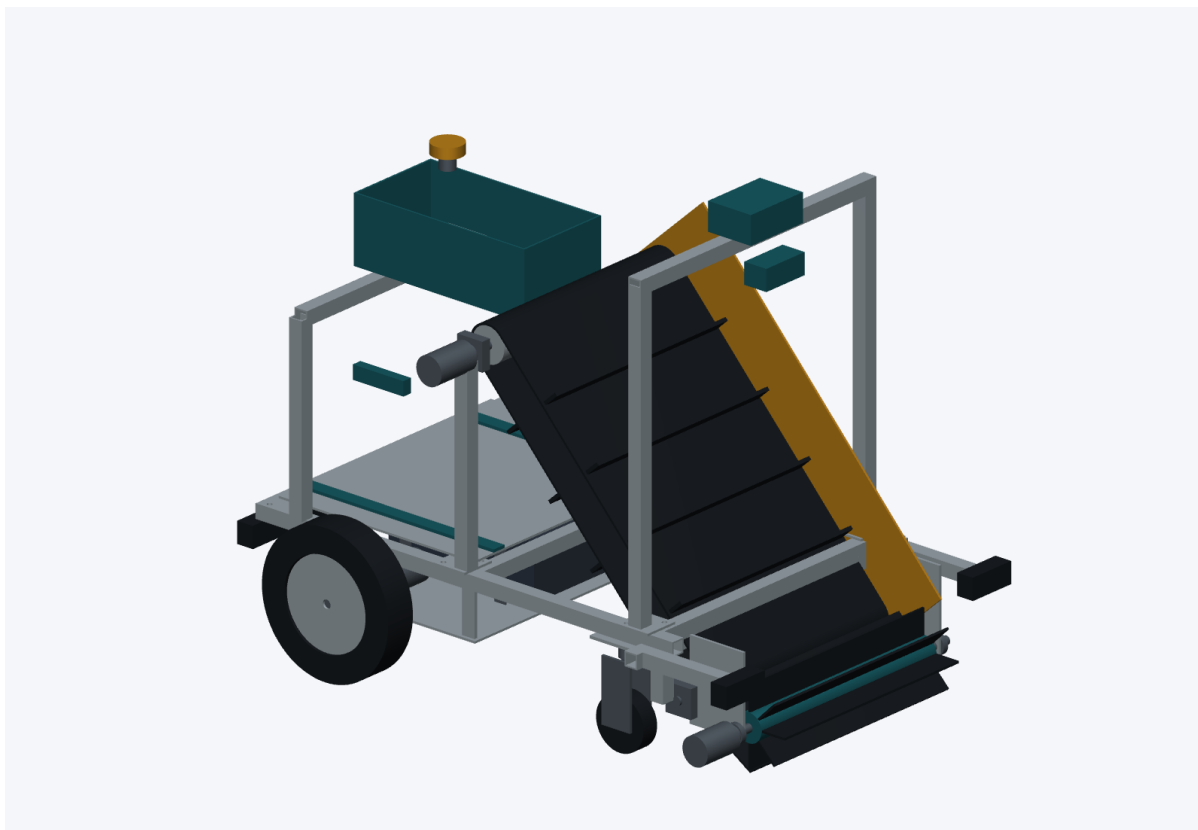


Рисунок 3. Вид с убранными закрывающими элементами: несущая структура и расположение узлов.

Файлы STEP доступны большинству инженерных CAD, поддерживающих этот формат. Нативные SLDPRT/SLDASM не созданы, проверка открытия именно в SolidWorks не выполнялась. После импорта тела обычно не получают исходную историю параметрических операций CadQuery. Изменять базовую конструкцию надёжнее в исходнике и заново экспортировать STEP; локальная доработка в выбранной CAD допустима, но создаёт вторую ветвь геометрии, которую нужно версионировать.

Проверка валидности тел отвечает на вопрос о корректности топологии. Она не проверяет автоматически прочность, технологичность, крепление всех покупных изделий или отсутствие всех столкновений при движении. У гибкой щётки есть намеренный контакт с губой, а ременная передача идеализирована. Такие места требуют физической модели или стенда, а не простого запрета на пересечение твёрдых тел.

04. Тяга, скорость, масса и запас энергии

Расчёт тяги начинается с полной массы, а не с массы алюминиевой рамы. В R01 приняты 30 кг: сюда должны войти батарея, провода, крепёж, приводы, электроника и содержимое бункера. Оценочная масса смоделированных частей с некоторыми заменами на массы покупных узлов составляет 24,31 кг. При полезной нагрузке 4 кг остаётся около 1,69 кг до расчётного предела. Это небольшой резерв; его нельзя тратить повторно в нескольких подсистемах.

Для предварительного расчёта приняты коэффициент сопротивления качению 0,03, уклон 5° и ускорение 0,25 м/с². Это проектные предположения, не измерения. Сила на двигателях складывается из сопротивления качению, составляющей тяжести и инерции:

```
F = m · g · (sin α + c_r · cos α) + m · a
m = 30 кг; g = 9,81 м/с²; α = 5°; c_r = 0,03
F ≈ 41,95 Н
T_одного_колеса = (F / 2) · r · 1,5
r = 0,10 м; T ≈ 3,15 Н·м
```

Множитель 1,5 — выбранный предварительный запас, а не универсальное нормативное требование. Для дальнейшего подбора задано не менее 4 Н·м длительного момента на выходе каждого редуктора при скорости около 30 об/мин под нагрузкой. Нельзя заменить длительный момент моментом заклинивания или принять рекламные «6 А» драйвера за допустимый бесконечный ток системы. Проверяется вся цепь: мотор, редуктор, муфта, вал, подшипники, колесо и охлаждение драйвера.

При скорости 0,30 м/с колесо радиусом 0,10 м вращается с частотой примерно 28,65 об/мин. Расчёт говорит, что редуктор со скоростью 300 об/мин потребует другого управления и существенно большего запаса по минимальной устойчивой скорости. Он не становится эквивалентным варианту 30 об/мин просто после ограничения PWM до десяти процентов: меняются момент, нагрев и старт под нагрузкой.

Аккумулятор в компоновке — готовая защищённая сборка 8S LiFePO4, номинально 25,6 В и 10 А·ч. Номинальная энергия равна 256 Вт·ч. При условных 80 Вт средней нагрузки, 80% используемой энергии и 90% эффективности цепи получается 2,304 часа. Это сценарная оценка. На реальной машине средний ток щётки, ленты, камеры и вычислителя нужно записать за полный цикл и повторить расчёт по измеренному профилю.

```
E_nom = 25,6 · 10 = 256 Вт·ч
t_оценка = 256 · 0,8 · 0,9 / 80 ≈ 2,30 ч
```

Для одной балки профиля 20 × 20 × 2 мм момент инерции сечения равен 7872 мм⁴. В принятой простой балочной схеме исходник получает напряжение около 30,84 МПа и прогиб 1,62 мм. Этот расчёт полезен как сигнал порядка величин, но не заменяет проверку соединений, вырыва отверстий, сварного шва, усталости и концентраций напряжений. Если рама изготовлена из материала с другой маркой и термообработкой, нельзя переносить допустимые напряжения без проверки.

05. Щётка, лента и геометрия приёма

Передний подборщик выбран вместо манипулятора, чтобы уменьшить число управляемых степеней свободы. Машина сама подводит приёмное окно к предмету. Гибкая щётка помогает завести предмет на губу и нижний участок ленты. Лента переносит предмет выше борта бункера и сбрасывает внутрь. Эта схема проще руки с захватом, но хуже выбирает один предмет из тесной группы и чувствительна к форме покрытия.

Ширина рабочего канала — 320 мм. Ось щётки расположена около $X = 383$ мм, $Z = 62$ мм, наружный радиус гибких лопастей — 57 мм. Губа подборщика подходит к полу примерно на 4 мм. Малый зазор облегчает подбор бумаги, но увеличивает риск зацепа за шов плитки. Поэтому первоначальный полигон должен быть гладким; имитация дорожного бордюра не входит в начальные испытания.

Оси роликов ленты: нижняя (300; 0; 85) мм, верхняя (−30; 0; 445) мм. Межосевое расстояние составляет 488,36 мм, угол наклона — 47,49°. Для расчётного радиуса средней линии 26,5 мм получаем длину замкнутой средней линии около 1143,23 мм:

$$C = \sqrt{(300 + 30)^2 + (445 - 85)^2} = 488,36 \text{ мм}$$

$$L \approx 2 \cdot C + 2 \cdot \pi \cdot 26,5 = 1143,23 \text{ мм}$$

Это не заказная длина промышленной ленты. Производитель может нормировать внутреннюю длину, длину по корду, допуск натяжения или шаг зацепления. Реальный материал растягивается, шов занимает место, а поверхность ролика может иметь покрытие. Перед заказом нужно выбрать тип ленты, натяжитель и способ крепления поперечных планок, затем пересчитать длину по правилам изготовителя.

В CAD поперечные планки и поверхность ленты идеализированы. При наклоне около 47,5° одной силы трения может не хватить: предмет будет скатываться. Поэтому нужны экспериментальные прогоны с бумагой, бутылкой и банкой разных размеров. Результат фиксируется отдельно для входа на ленту, удержания между планками и сброса в бункер. Один показатель «подобрано» скрывает разные причины отказа.

Входной лучевой датчик подтверждает пересечение приёмной зоны, выходной — выгрузку. По одному наличию изображения нельзя решить, что предмет уже лежит в бункере. В R01 после подтверждённого входа машина останавливает шасси и довозит уже принятый предмет лентой. По истечении времени переноса без выходного подтверждения процесс завершается отказом, а не бесконечным вращением.

Руки и одежда не должны иметь доступа к зоне заземления. Перед реальным запуском требуются закрытые передачи, защита торцов валов, проверка зазоров по всей траектории и работающая блокировка кожуха. Внешний силуэт CAD не является доказательством защищённости всех опасных точек. Испытание щётки выполняется с безопасными макетами и инструментом подачи, без удерживания предмета пальцами перед вращающимися лопастями.

06. Сборка, обслуживание и размерные цепи

Сборку начинают с неподвижной рамы на плоском столе. Проверяют диагонали, параллельность продольных элементов и взаимное положение опор ведущей оси. Нельзя компенсировать перекошенную раму регулировкой программного коэффициента левого колеса: это создаёт боковые нагрузки и ухудшает повторяемость движения. Сначала механика должна свободно работать без программной коррекции.

Следующий узел — ведущие валы с опорами. В R01 их положение задано геометрически, но окончательные посадки, стопорение и каталожные подшипники не выбраны. Вал не должен удерживаться от осевого перемещения только трением случайной муфты. Для рабочего выпуска выбирают опорную схему, подшипники, допуски, стопорные элементы и крепёж, после чего обновляют CAD и спецификацию.

Батарею устанавливают на нижнюю опору и фиксируют от смещения во всех направлениях. Упругий ремень не должен перетираться об острый профиль. Провод аккумулятора защищают в месте выхода, главный предохранитель располагают близко к источнику энергии. Передняя механика и контактор ещё обесточены. Мощный аккумулятор не используется как удобный источник для первых опытов с неизвестной платой драйвера.

Ленту собирают как самостоятельный модуль. Проверяют соосность роликов, боковое положение полотна, достаточный диапазон будущего натяжителя и отсутствие контакта планок с неподвижными щитками. В R01 натяжной узел требует детализации; он не должен появляться на настоящей машине «по месту» без отражения в новой ревизии. Непредсказуемое натяжение напрямую меняет ток и нагрев привода.

Бункер опирается на отдельную площадку и извлекается назад. Его наружные размеры 320 × 360 × 220 мм, толщина стенок и дна в геометрической модели — 3 мм. Внутренний объём вычисляется как 314 × 354 × 217 мм³, то есть около 24,12 л. Полезный объём меньше: зона падения, датчик заполнения и неравномерное распределение предметов требуют свободного места.



Рисунок 4. Вид сзади. Для обслуживания сохраняется отдельное направление извлечения бункера.

Окончательную массу и положение центра тяжести проверяют после монтажа проводки. Загруженный бункер может сместить центр тяжести существенно выше и назад. Нельзя считать батарею универсальным противовесом только потому, что она установлена низко. Для проверки устойчивости нужны реальные опорные точки колёс, измеренная масса и медленные стендовые испытания, исключающие неконтролируемое опрокидывание.

Комплект содержит два DXF плоских элементов и эскизный альбом проекций, но не полный набор раскроя и рабочих чертежей всех 121 тел. До изготовления необходимо выпустить спецификацию крепежа, посадок, гибов, способов соединения, натяжителя и защит. Исходная модель полезна именно как согласованная компоновка, которую можно последовательно довести до такого выпуска.

07. Электропитание: от батареи до двигателя

Силовая система разделена на логическую и моторную ветви. После главного предохранителя F1 и разъединителя S0 логическая ветвь через F2 питает преобразователь 5,1 В. Моторная ветвь проходит через контактор K1 к общей шине VM_BUS и далее через индивидуальные предохранители к четырём драйверам. Отключение приводов не должно одновременно уничтожать журнал и связь с пультом; поэтому Raspberry Pi питается до K1.

В проекте задан преобразователь с выходом 5,1 В / 5 А и входом, рассчитанным не менее чем на рабочий диапазон батареи. Питание Raspberry Pi 5 заведено через USB-C. В схеме JUSB1 обозначает функциональный интерфейс питания, а не готовую разводку разъёма USB-C с линиями конфигурации. Конкретный источник должен корректно поддерживать требуемый режим питания Pi; одной надписи «5 А» на DC/DC для этого недостаточно. [4]

Номинал батареи 25,6 В не является её максимальным напряжением. Для полностью заряженной 8S-сборки в проектных проверках используется 29,2 В. Напряжение повышается также при возврате энергии от мотора. Поэтому запас по напряжению нужно оценивать для реального режима остановки, а не только для неподвижного заряженного аккумулятора. Входной разъединитель и контактор выбираются с допустимой коммутацией постоянного тока, а не по сходному числу ампер в цепи переменного тока.

В VM_BUS включён конденсатор 10000 мкФ / 50 В. Его энергия при 29,2 В составляет около 4,26 Дж. Подключение незаряженного конденсатора напрямую через контактор может дать большой бросок тока. Для стендовой версии предусмотрен ручной предварительный заряд через 22 Ом: удерживать SPRE не менее 1,2 с, затем выполнить аппаратный ARM и отпустить SPRE. Резистор должен выдерживать импульсную энергию не менее 5 Дж по документации конкретного изделия; фактический режим проверяется измерением.

$$\begin{aligned}\tau &= R \cdot C = 22 \cdot 0,010 = 0,22 \text{ с} \\ 5\tau &\approx 1,10 \text{ с} \\ E_C &= C \cdot V^2 / 2 \approx 4,26 \text{ Дж при } 29,2 \text{ В}\end{aligned}$$

Эта цепь предварительного заряда — ограничение прототипа, а не законченный промышленный автомат. Нажатый SPRE соединяет батарею с моторной шиной через резистор даже при разомкнутом K1. Следовательно, K1 нельзя считать единственным средством безопасного изолирования при обслуживании. Сервис начинается с S0, отключения источника и проверки остаточного напряжения, а не с нажатия экранного STOP.

Разрядный резистор 1 кОм / 2 Вт уменьшает остаточное напряжение, но не делает это мгновенно: постоянная времени с указанной ёмкостью около 10 с. При снятии крышки нельзя полагаться только на истечение условной паузы. Напряжение измеряется на предусмотренных контрольных точках. Также проверяется, не остаётся ли второй путь питания от лабораторного блока или подключённой периферии.

Главный незавершённый вопрос силовой части R01 — гарантированное ограничение рекуперативного перенапряжения. Конденсатор не заменяет расчёт тормозной энергии и

зажим напряжения. Надпись «TVS 33 В» сама по себе не обеспечивает предел ниже максимально допустимого для драйвера: важно реальное напряжение ограничения при заданном токе. До движения на батарее необходимо выбрать и испытать решение: управляемое торможение, поглощение энергии, допустимый путь возврата в батарею и защиту при отключении BMS. Прототип без этой проверки не выпускается в моторные испытания.

08. Четыре одинаковых канала управления мотором

Два двигателя вращают ведущие колёса; третий приводит щётку; четвёртый — ленту. Для каждого предусмотрен канал на DRV8874 с режимом PH/EN. Направление формируется расширителем SPI, заполнение PWM — внешним генератором I²C. Разделение позволяет Java выдавать медленные команды без программного формирования каждого импульса на GPIO. Драйвер допускает соответствующее управление логическими уровнями, но его пиковый ток нельзя трактовать как гарантированный длительный ток готовой платы. [5]

PCA9685 работает на номинальной частоте около 1017 Гц при выбранном делителе 5 и внутреннем генераторе. Частота не составляет 20 кГц и находится в слышимом диапазоне. Выбор сделан ради простой четырёхканальной реализации. После испытаний её можно изменить только с проверкой диапазона периферии, шумов, нагрева и характеристик мотора. Эта микросхема является генератором PWM, а не силовым драйвером. [6]

Канал 0 — левое колесо, 1 — правое, 2 — щётка, 3 — лента. GPA0...GPA3 MCP23S17 задают PH тех же каналов. Эта нумерация повторяется в Java, схеме, АЦП токов и таблице соединений. Перестановка двух моторов при сохранении старых знаков энкодеров может превратить отрицательную обратную связь в положительную. На стенде каждый канал испытывается отдельно, с вешенными колёсами и ограниченным источником тока.

Порог аппаратного ограничения тока задаётся VREF. В расчётной схеме принят коэффициент IPROPI 455 мкА/А и резистор 1,5 кОм, что даёт примерно 0,6825 В/А. Делитель 6,19 кОм / 10 кОм формирует около 2,04 В для приводов хода, а 14,3 кОм / 10 кОм — около 1,36 В для механизмов. Получаются ориентиры примерно 3 А и 2 А. Это расчёт настройки, а не измеренная точность ограничения на собранной плате. [5]

Между IPROPI и входом АЦП установлен фильтр 10 кОм / 100 нФ. Он подавляет часть пульсаций, но измеренный сигнал зависит от PWM и режима циркуляции тока. В коде предусмотрены более низкие программные пороги с выдержкой времени. Перед использованием их нужно сопоставить с током, измеренным подходящим прибором, при разных нагрузках. Программное обнаружение заклинивания не заменяет быстросействующее ограничение драйвера.

Конденсаторы зарядового насоса соединяются строго по назначению выводов: 22 нФ между CPH и CPL; 100 нФ между VCP и VM, а не между VCP и землёй. На локальной моторной шине предусмотрены керамический и электролитический конденсаторы. Их наличие в схеме не доказывает, что длинный провод от батареи, индуктивность дорожек и токовая петля будут приемлемы. Печатная плата в R01 не разведена, поэтому тепловая и высокочастотная работа остаётся предметом отдельного проектирования. [5]

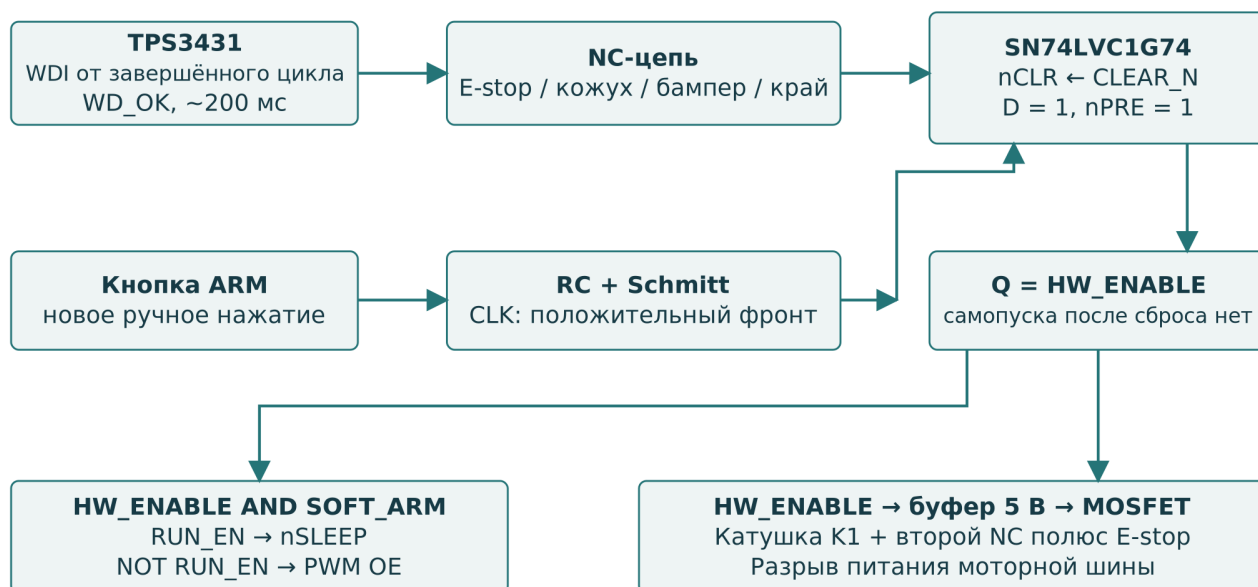
При изменении знака команды программный байтовый драйвер сначала отключает PWM, затем меняет PH и возвращает заполнение. Это логическая последовательность, а не доказанная аппаратная пауза переключения транзисторов. Плавность изменения команды дополнительно ограничивает DriveControl. Непосредственное мгновенное реверсирование тяжёлого привода при большой скорости не является допустимым способом аварийной остановки этой версии.

09. Аппаратный останов, который не зависит от нейросети

Самая опасная архитектурная ошибка — считать периодическое выполнение отдельного таймера признаком исправности всего управления. Если поток камеры завис, а независимый поток heartbeat продолжает работать, аппаратная сторожевая схема получает ложное подтверждение. В R01 heartbeat меняется только после успешно завершённого цикла чтения, решения и записи приводов. Ошибка обмена или зависание этого пути прекращают подтверждение.

TPS3431 контролирует heartbeat на WDI. Для выбранного включения CWD через 10 кОм к питанию и SET1 в высоком уровне используется номинальный интервал 200 мс с диапазоном порядка 170–230 мс. При цикле 50 Гц GPIO меняет состояние каждые 20 мс, поэтому контролируемые отрицательные фронты следуют примерно каждые 40 мс. Эти числа относятся к проектной конфигурации; фактическую задержку отключения контактора необходимо измерить. [7, 8]

Выход WD_OK проходит через последовательную цепь нормально замкнутых контактов аварийной кнопки, кожуха, бампера и сигнала опасного края. Разрыв любого участка снимает CLEAR_N у триггера SN74LVC1G74. Его Q становится аппаратным разрешением HW_ENABLE. Восстановление контакта или heartbeat не устанавливает Q автоматически: нужен новый положительный фронт на физической кнопке ARM. Удерживаемая кнопка не должна обойти требование повторного пуска. [9]

UBOR-JAVA R01 / аппаратное разрешение и повторный пуск

Проектный контур. Не заявлены PL/SIL, измеренная задержка K1 или выполненное SPICE-испытание.

Рисунок 5. Логика разрешения: сброс может возникнуть независимо, восстановление требует нового ручного фронта.

D триггера и неактивный PRE установлены в высокий уровень. Физическая кнопка проходит RC-цепь и триггер Шмитта, чтобы уменьшить влияние дребезга. Начальный сброс использует предусмотренное в сторожевой схеме включение ENOUT. Всё это задано как проект соединений. Переходные процессы включения питания, медленное нарастание 3,3 В, дребезг реальной кнопки и задержка реле не проверялись SPICE или осциллографом и должны войти в стендовый протокол.

HW_ENABLE включает катушку K1 через буфер SN74AHCT1G125 с питанием 5 В и транзистор FQP30N06L. Пятивольтовой амплитуде затвора соответствует документированный режим транзистора; прямое управление этим затвором от GPIO 3,3 В здесь не заявляется как эквивалентное. Диод на катушке подавляет выброс, но может увеличить время отпускания. Это ещё одна причина измерять полную задержку размыкания, а не останавливаться на таймауте watchdog. [10, 11]

Есть и программное разрешение SOFT_ARM. Только логическое И SOFT_ARM с HW_ENABLE формирует RUN_EN для nSLEEP драйверов. Инверсия RUN_EN запрещает выходы PWM через OE. Таким образом, отказ одного программного уровня не должен автоматически разрешить силовой выход. Однако вся схема не имеет заявленного уровня PL/SIL и не заменяет анализ отказов, подходящий для публичной или промышленной эксплуатации.

Контакты, передающие состояние на Pi, должны быть электрически отделены от контактов силового или аппаратного разрешения. Один и тот же физический вывод нельзя одновременно включить в цепь 24 В и подключить к GPIO. Для аварийной кнопки требуются отдельные полюса: разрыв катушки, аппаратная цепь разрешения и считывание состояния. В KiCad эти функции обозначены отдельно; конкретное устройство с нужным количеством контактов ещё нужно выбрать.

10. Шины, GPIO и электрические интерфейсы

Каждая линия в `electronics/wiring.csv` имеет имя цепи, обозначение компонента, номер вывода и имя сигнала. Это каноническая таблица соединений. В иерархическом KiCad одинаковые глобальные метки объединяют цепь между листами. Схемы специально самодостаточны по условным символам. Их структурная проверка не равна ERC: выводы абстрактных модулей не несут полной информации о типах питания и допустимых токах.

В заголовке Pi используются физические номера контактов, а в исходниках — BCM. Нельзя перенести число 22 из столбца BCM прямо в номер штырька: GPIO22 находится на физическом контакте 15. Перед монтажом печатают таблицу, маркируют разъём и проверяют каждую цепь прозвонкой при снятом питании. Неиспользуемые контакты питания не соединяют «для надёжности» с другими источниками.

Функция	BCM	Физический контакт	Сигнал
I ² C SDA / SCL	2 / 3	3 / 5	I2C_SDA / I2C_SCL
SPI0 MOSI / MISO	10 / 9	19 / 21	SPI0_MOSI / SPI0_MISO
SPI0 SCLK / CS0 / CS1	11 / 8 / 7	23 / 24 / 26	ADC и направления
SPI1 MOSI / MISO	20 / 19	38 / 35	Счётчики энкодеров
SPI1 SCLK / CS0 / CS1	21 / 18 / 17	40 / 12 / 11	Счётчики энкодеров
Heartbeat / Soft arm	22 / 27	15 / 13	HEARTBEAT / SOFT_ARM
Аварийный / Бампер	23 / 24	16 / 18	ESTOP_SENSE / BUMPER_SENSE
Край / Кожух	25 / 5	22 / 29	CLIFF_SENSE / GUARD_SENSE
Заполнение	6	31	FULL_SENSE
Вход / Выгрузка	12 / 13	32 / 33	INTAKE_BEAM / DROP_BEAM
Ошибка драйвера / K1	16 / 26	36 / 37	DRV_FAULT_N / K1_SENSE

Для контактов безопасности нормальное состояние соответствует LOW: исправный нормально замкнутый контакт соединяет вход с землёй, внешний резистор 10 кОм поднимает оборванный вход в HIGH. Ошибка драйвера `nFAULT`, напротив, активна низким уровнем. Лучевые датчики также дают LOW при прерывании луча. Эти различия отражены в PiHardware, и их нужно проверить на стенде индивидуально.

MCP3008 использует CS0 шины SPI0. CH0 измеряет батарею через делитель 100 кОм / 10 кОм. CH1...CH4 измеряют токи каналов 0...3. CH5 принимает напряжение аналогового дальномера. CH6 и CH7 соединены с землёй. Расширитель MCP23S17 находится на CS1.

Его адресные входы зафиксированы в нуле; четыре младших вывода порта А задают направления, остальные входы подтягиваются программно. [12, 13]

Два LS7366R расположены на SPI1 с отдельными CS. Их задача — считать квадратурные импульсы без зависимости от задержки Java-потока. В схеме предусмотрен внешний генератор 1 МГц с логикой 3,3 В. Текущий коэффициент программы — 2048 эффективных отсчётов на оборот колеса после квадратурного умножения. Это требование к выбранному энкодеру и редуктору, а не универсальный параметр любых моторов. [14]

Дальномер задан через интерфейс безопасного напряжения 0...3,3 В. Конкретная модель и характеристика не выбраны. Файл `range-calibration.csv` поэтому пуст: загрузчик отказывается запускать аппаратный режим без измеренной монотонной таблицы. Подстановка правдоподобной кривой чужого датчика была бы хуже честного запрета запуска. Аналогично датчик края должен иметь выбранную и проверенную схему формирования безопасного контакта; одной метки CLIFF в проекте недостаточно для эксплуатации рядом с лестницей.

11. Java-ядро: данные, свежесть и состояния

В `Types.java` определены неизменяемые записи команд, датчиков, результата восприятия, решения и сигналов приводов. На входе проверяются диапазоны и конечность чисел. NaN опасен тем, что обычное сравнение может не сработать ожидаемым образом; поэтому недопустимое число отвергается до расчёта PWM. Метки времени получены монотонным таймером, а не календарными часами компьютеров.

У команды есть ограниченное время действия. Командный канал не задаёт режим «ехать, пока когда-нибудь не придёт STOP». Для продолжения движения нужны свежие подтверждения удержания `deadman`. В R01 срок команды — 200 мс, датчиков — 150 мс, зрения в автоматическом режиме — 250 мс. Эти величины выбраны для прототипа и должны подтверждаться измерениями полной задержки, включая камеру и сетевой тракт.

`SafetyController` содержит состояния `DISARMED`, `TELEOP`, `AUTO`, `COLLECT` и `FAULT`. Он проверяет аппаратные блокировки раньше обычной команды движения. Ряд ошибок фиксируется до явного сброса: аварийный останов, недостоверные или устаревшие датчики, превышение времени цикла, ошибка драйвера, низкое напряжение и длительная перегрузка. Исчезновение причины само по себе не означает разрешение продолжить поездку.

Сброс выполняется при отпущенном `deadman` и снятой программной команде `arm`. Отдельно существует физический повторный пуск аппаратного разрешения. Эти два действия не дублируют друг друга: одно относится к состоянию Java-контроллера, второе — к разрешению питания. Пульт не заменяет кнопку аппаратного ARM.

Автоматический режим принимает цель только при достаточной уверенности классификатора. В R01 порог допуска 0,90, порог блокирующей опасной оценки — 0,60. Это начальные инженерные настройки, а не статистически обоснованные пороги безопасности. Их нужно выбирать по отдельной проверочной выборке с учётом цены ошибок, а не по привлекательной точности на обучающем наборе.

Особый переход связан с переносом предмета. До входного луча система ещё принимает решение, подходит ли цель. После подтверждённого входа цель считается принятой, шасси останавливается, щётка выключается, лента довозит предмет до выхода. Если в это время камера увидела следующий объект на полу, его обычная переоценка не должна отменять доставку уже принятого предмета. Но аппаратный аварийный сигнал, недостоверный датчик или другая блокировка продолжает отключать приводы.

Этот переход появился после фактически найденного интеграционного сбоя: распознавание следующего объекта останавливало транспортёр до выгрузки предыдущего. Исправление проверено отдельным утверждением и общим модельным прогоном. Оно показывает пользу явного автомата состояний: причина решения видна в телеметрии, а не скрыта внутри последовательности случайных `if`.

12. Запускаем собранный симулятор

Готовый `software/dist/ubor-sim.jar` не требует `Pi4J`, `DL4J`, камеры или `Maven` для запуска. Нужна `Java 21`. Сначала проверяют `java -version`, затем переходят в каталог `software`. На `Windows` команды выполняются в `PowerShell` или терминале `IntelliJ`. Полный `JDK` понадобится для пересборки; для запуска уже собранного `JAR` достаточно подходящей среды выполнения.

```
java -version
java -jar dist/ubor-sim.jar --desktop
```

В открывшемся `Swing`-пульте видны полигон, синтетический вид камеры и телеметрия. Режим явно обозначен `SIM_ORACLE_NOT_ML`. В симуляторе местоположение и класс предметов берутся из внутреннего состояния мира. Это полезно для отладки управления и переноса, но не является моделью обученного зрения.

Для ручного движения нажимают `Arm / Disarm` и удерживают `SPACE`. `W` или стрелка вверх задаёт движение вперёд; `A/D` или боковые стрелки — поворот; `S` — медленное движение назад. Кнопка `C` включает подбор с выбранной человеком меткой допустимого предмета. При потере фокуса окна сбрасывается разрешение. `SPACE` не является переключателем: его необходимо держать.

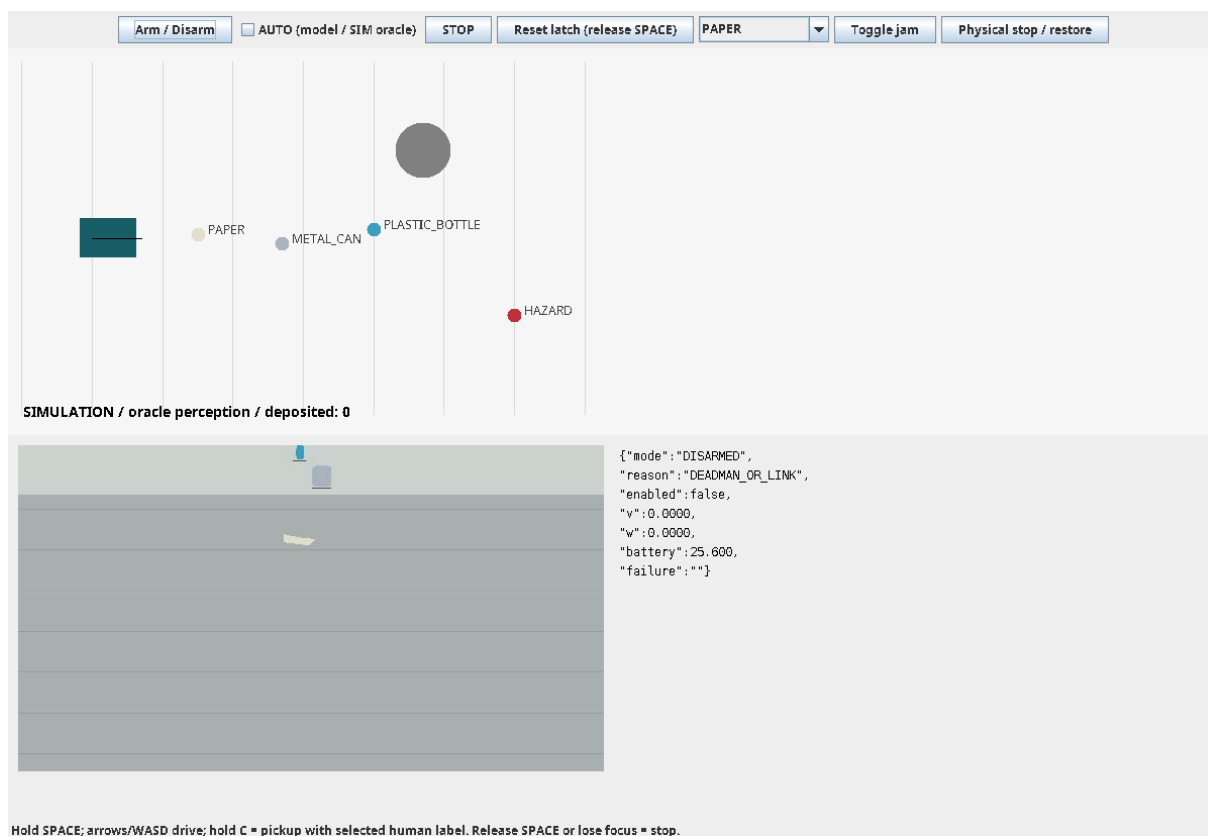


Рисунок 6. Снимок действительно запущенного Swing-пульта под Xvfb: карта, синтетический кадр и телеметрия.

Для воспроизводимого прогона без интерфейса используется команда ниже. Она моделирует 90 секунд, записывает координаты, карты и итоговый JSON. Прогон может выполняться быстрее реального времени, поэтому время симуляции нельзя сравнивать с замером производительности на Raspberry Pi.

```
java -jar dist/ubor-sim.jar --demo ../verification/my-demo 90
java -jar dist/ubor-sim.jar --self-test ../verification/my-tests
```

Исходники core/app пересобираются без внешних библиотек. Скрипт выполняет javac, создаёт JAR и запускает проверки. Обычный Maven-проект также подготовлен, но в среде создания данного комплекта Maven и загрузка зависимостей были недоступны. Это означает, что результат офлайн- сборки нельзя объявлять подтверждением полного профиля full.

```
# Linux / macOS с JDK 21
bash build-offline.sh

# Windows PowerShell с JDK 21
powershell -File build-offline.ps1

# Альтернативная сборка: требует установленный Maven
mvn clean verify
```

Симулятор содержит упрощённую динамику колёс, координаты машины, предметы, лучевые события, перенос по ленте и модель отключения watchdog. Он не рассчитывает деформацию бутылки, контакт всех CAD-поверхностей, ток в транзисторах или реальную инерцию барабана. Сцена из CAD и логический полигон связаны размерами и функциями, но не объединены в физический многотельный решатель.

13. Цикл управления и регуляторы колёс

RobotService создаёт отдельный исполнитель для цикла управления с периодом 20 мс. В одном успешном проходе он читает аппаратные входы, получает актуальную команду и последний результат восприятия, вычисляет решение, формирует воздействия и записывает их в аппаратный интерфейс. Только после удачного завершения записи меняется heartbeat. Запись PNG, отправка предпросмотра и нейросеть не выполняются в этом потоке.

Вторая задача отвечает за кадры, инференс и запись учебных данных. Она может работать медленнее 50 Гц: управляющий поток пользуется последним результатом, пока он достаточно свеж. Если камера или сеть задержалась настолько, что результат устарел, автоматический режим останавливается. Так устраняется требование ждать окончания нейросети перед каждым решением о состоянии приводов.

В модели датчики получают монотонную метку времени. В аппаратном адаптере чтение шин занимает ненулевое время, поэтому отметка берётся консервативно относительно начала чтения. Для команды время задаёт принимающая сторона. Не требуется синхронизировать календарные часы пульта и Pi для проверки lease; системное изменение времени или разный часовой пояс не должны продлить старую команду.

Дифференциальная кинематика для колеи $B = 0,5$ м записывается так:

```
v_left  = v - ω·B/2
v_right = v + ω·B/2
Ω_left  = v_left / r
Ω_right = v_right / r
```

DriveControl сочетает предварительную оценку заполнения по скорости и PI-коррекцию по энкодерам. Интегральная составляющая ограничена, чтобы не накапливать огромную ошибку при насыщении. Скорость изменения воздействия также ограничена. При запрете движения выход выключается непосредственно; обычное плавное снижение команды не должно задерживать аварийный запрет.

Параметры регулятора — начальная настройка для прототипа, не результаты идентификации выбранного мотора. На вывешенном колесе проверяют знак, затем малую скорость и переходный процесс. На полу повторяют измерение с пустым и нагруженным бункером. Слишком большие коэффициенты могут дать колебание, а слишком маленькие — заметное отклонение и плохой поворот. Уменьшение колебаний программным фильтром не оправдывает неправильный знак обратной связи.

Отдельный незавершённый пункт R01 — диагностика правдоподобия энкодера. Байт счётчика корректно читается и переполнение учитывается, но полноценный контроль оборванной линии, невозможного скачка и несоответствия движению ещё не реализован. До реальных автоматических поездок нужно добавить и испытать такой монитор. Исправный SPI-пакет со счётчиком ноль не доказывает неподвижность колеса.

При исключении аппаратного обмена управляющий цикл прекращает обновление heartbeat, пытается выключить выходы и фиксирует ошибку. Нельзя обещать, что попытка программного отключения обязательно дойдёт по неисправной шине. Поэтому независимый watchdog является частью конструкции, а не дополнительным удобством.

Ни наличие `ScheduledExecutorService`, ни выбор Java 21 не превращают Linux в систему жёсткого реального времени.

14. Пульт, геймпад и управление по сети

Пульт написан на Java Swing. Один сеанс оператора использует TCP: сервер выдаёт случайный challenge, клиент подписывает команды HMAC-SHA256. Порядковый номер защищает от повторения старого пакета в том же сеансе, challenge — от переноса пакета в новый. Протокол ограничивает размеры сообщений и отвергает неполные или повреждённые данные.

Подписана именно команда управления. Телеметрия и JPEG-предпросмотр не защищены отдельной криптографической подписью и не шифруются. Поэтому этот протокол не предназначен для открытого Интернета. На прототипе используется изолированная доверенная сеть. Для реального сетевого продукта потребуются аутентифицированное шифрование, управление ключами, политика доступа и анализ отказов. Слово HMAC не означает, что все сетевые риски уже устранены.

Секрет не зашит в исходники. Утилита генерирует новый случайный ключ; одно и то же значение задаётся через `UBOR_KEY` на машине и пульте. Генерировать разные ключи независимо на двух сторонах нельзя: связь закономерно не пройдёт проверку. Ключ не записывается в репозиторий и не прикладывается к публичному отчёту.

```
# На одной стороне, затем безопасно передать значение другой
java -jar dist/ubor-sim.jar --new-key
```

```
# В обоих терминалах задать ОДИН и тот же полученный секрет
export UBOR_KEY='сюда_сгенерированное_значение'
```

```
# Сервер-симулятор
java -jar dist/ubor-sim.jar --server-sim 9090
```

```
# Пульт на другом компьютере
java -jar dist/ubor-sim.jar --controller 192.168.10.20 9090
```

В PowerShell переменная задаётся как `$env:UBOR_KEY='...'`. Пульт обновляет команды примерно 20 раз в секунду. Сервер не сохраняет разрешение бесконечно: отсутствие действительного пакета закрывает сеанс, а короткая аренда команды в контроллере служит дополнительным ограничителем. Автоматического перепоключения с восстановлением ARM нет. После потери связи оператор должен заново установить управление.

Для Linux реализован чтение устройства `/dev/input/js0` непосредственно в Java: 8-байтовые события `joydev` разбираются с нужным порядком байтов. Используются оси 0 и 1, кнопка 4 — `deadman`, 0 — `arm` без удержанного `deadman`, 1 — `disarm`, 2 — переключение `AUTO` без `deadman`, 3 — `STOP`, 6 — сброс. Инициализационные события не должны сами включать машину. Эта раскладка является проектной; конкретный USB-геймпад физически не испытан. [15]

```
java -jar dist/ubor-sim.jar \
  --gamepad 192.168.10.20 9090 /dev/input/js0
```

Геймпад в текущей версии не имеет интерфейса ручной разметки объекта и ручного запуска подборщика с меткой. Для таких действий используется Swing-пульт. Автоматический подбор с геймпада возможен только в режиме с допущенной моделью или в явно обозначенном ocase-симуляторе. Показанный на экране STOP полезен, но остаётся сетевой командой и не заменяет физическую кнопку на машине.

15. Pi4J: от Java-объекта к байтам на проводе

Pi4J 3.0.4 выбран сознательно, а не как автоматически «самая новая версия». Ветка 3 требует Java 21; выпуск 3.0.4 содержит исправление границ цикла SPI. Более новая ветка 4 имеет другой базовый уровень Java. При выпуске изделия версии фиксируют вместе, а обновление проводят как отдельное изменение с повторным стендовым тестом. [1]

Аппаратный модуль использует GpioD для цифровых линий и LinuxFS для SPI/I²C. В PiHardware собираются конфигурации соответствующих устройств. Однако доступность /dev/spidev..., нумерация GPIO chip и права зависят от реально установленной ОС. Одного наличия JAR недостаточно: интерфейсы нужно включить и проверить в системе, а пользователя сервиса включить в необходимые группы. В этой среде такие проверки на Pi не проводились. [2]

Байтовые драйверы находятся в независимом RegisterDevices.java. Например, чтение канала MCP3008 выполняется трёхбайтовой транзакцией. Такое разделение позволяет тестировать формирование команды без электрической платы:

```
byte[] rx = bus.transfer(new byte[]{
    1, (byte)((8 + channel) << 4), 0
});
int value = ((rx[1] & 3) << 8) | (rx[2] & 255);
```

Здесь & 255 устраняет знаковую интерпретацию Java-байта. Тест подтверждает порядок байтов и декодирование, но не частоту SPI, корректность уровней или отсутствие наводки. На первом стенде логическим анализатором сверяют CS, SCLK, MOSI и MISO с ожидаемой последовательностью. Подключение правильного устройства к неправильному CS может давать правдоподобные нули, поэтому проверяют каждый канал независимо.

Для PCA9685 важна особенность общей записи ALL_LED: это не независимый «главный разрешающий бит», который можно свободно очищать после индивидуальных каналов. Общая запись влияет на регистры каналов. В R01 после записи отдельных заполнений не выполняется дополнительное очистительное действие ALL_LED, которое могло бы испортить уже записанные значения. Эта последовательность выделена в отдельную проверку. [6]

LS7366R отдаёт четырёхбайтовый счётчик старшим байтом вперёд. При накоплении перемещения намеренно используется знаковая 32-битная разность для обработки переполнения. Чтобы это было корректно, между чтениями не должно пройти более половины диапазона счётчика. На скоростях R01 это большой запас, но само математическое предположение всё равно следует понимать, особенно при изменении частоты энкодера. [14]

Конфигурация блокирует аппаратный запуск до калибровки. Значения `motor.sign` и `encoder.sign` должны быть ± 1 и соответствовать проверенному направлению. Таблица дальномера должна содержать не менее четырёх реально измеренных монотонных точек. Для автоматического зрения отдельно проверяется `camera.calibration.complete`. Эти флаги служат запретом преждевременного запуска, но не способны проверить, действительно ли человек выполнил измерения: ответственность за процедуру остаётся у выпускающего инженера.

16. Камера, координаты и границы машинного зрения

В R01 выбрана UVC-камера с доступом через JavaCV/OpenCV. Начальная конфигурация — 640 × 360 при запрошенных 15 кадрах/с. Запрошенный режим не означает гарантированную частоту конкретной камеры: драйвер может согласовать другой формат, а очередь кадров может добавить задержку. Проверяется фактическое изображение и время между событием перед камерой и доступностью соответствующего кадра в Java.

Предобработка одинакова при обучении и эксплуатации: RGB, размер 64 × 64, float в диапазоне 0...1, порядок NCHW. Несогласованность RGB/BGR или другой масштаб входа может разрушить результат без исключения в программе. Поэтому преобразование вынесено в один класс `Images`, а формат записывается в метаданные модели.

Для локализации применяется базовый алгоритм скользящих окон двух масштабов в нижней области изображения. Каждое окно передаётся классификатору. Выбирается допустимый кандидат с лучшей оценкой; опасная оценка блокирует движение. Это не YOLO, не сегментация экземпляров и не полноценный детектор с регрессией координат. Крупная бутылка может попадать сразу в несколько окон, а часть фона — давать высокую уверенность по ошибке.

Начальная локализация использует нижнюю точку окна и пересечение соответствующего луча с плоскостью пола. Для этого нужны фокусные расстояния f_x/f_y , главная точка s_x/s_y , высота камеры, угол наклона и смещение до приёмного окна. Числа в `robot.properties` — оценки компоновки, не калибровка. На настоящей машине сначала измеряют внутренние параметры и искажения, затем внешний наклон и положение относительно пола.

Рассуждение о проекции простое: один пиксель задаёт луч, а плоскость пола выбирает на луче конкретную точку. Если предмет находится не на полу, а на ступени или верхняя часть бутылки используется вместо точки контакта, предположение нарушается. Поэтому в R01 не обещана точность геометрической локализации, достаточная для любого размера мусора. Её нужно измерить на разметке пола вблизи подборщика.

Классификатор и модель движения решают разные задачи. Классификатор получает вырезку предмета и отвечает классом. Модель движения получает полный кадр и предлагает v и ω , обучаясь по действиям человека. Нельзя обучить полнокадровую модель на маленьких вырезках и ожидать, что она поймёт маршрут. Также нельзя переносить метку «бумага» на весь сложный кадр, где одновременно находятся несколько объектов.

Высокая softmax-оценка не является гарантией распознавания неизвестного типа отхода. Класс UNKNOWN полезен как обучаемая категория примеров, но не решает автоматически проблему всех неизвестных изображений. Для реального выпуска требуется отдельно измерить ложные разрешения на неподходящих предметах, влияние бликов, теней, цвета пола, собственной щётки в кадре и частично скрытых объектов. До этого AUTO остаётся лабораторным режимом под непосредственным наблюдением.

17. Сбор данных при объезде

Возникает естественный вопрос: как получить первые данные, если аппаратный AUTO требует обученную модель? Для этого предусмотрен режим `--hardware-teleop`. Он использует настоящую камеру и ручное управление, но не требует весов нейросети. Аппаратная калибровка, блокировки и внешний watchdog при этом по-прежнему необходимы. Разница только в том, кто предлагает движение.

```
# После аппаратной проверки и сборки полного classpath
java -cp 'dist/full/*' com.ubor.app.Main \
  --hardware-teleop config/robot.properties 9090 ../data/run_01
```

DatasetRecorder сохраняет PNG и manifest.csv из потока камеры, а не из управляющего цикла. Запись допускается только при ручном режиме, удержанном deadman и действительном разрешении движения. Разница отметок кадра и команды должна быть не более 100 мс. Это необходимая начальная проверка согласования, но не измерение задержки экспозиции и очереди видеодрайвера.

Манифест содержит путь изображения, выборку, метку, применённые v/w , идентификатор сеанса, времена кадра и команды, исходно запрошенные скорости и причину решения. В обучении движения используются применённые команды: если SafetyController ограничил скорость, нельзя незаметно записать в качестве правильного действия запрещённую исходную величину. В то же время такие данные могут содержать остановки из-за блокировки; их смысл необходимо разбирать при формировании учебной задачи.

```
image,split,label,v,w,session,...
images/run01_000123.png,unassigned,PAPER,0.08,0.00,run01,...
```

Метка выбора оператора — не готовая разметка всех объектов кадра. Для классификатора дополнительно размечают прямоугольники. Утилита DatasetTool умеет вырезать их по CSV с полями image,x,y,width,height,label,session,split. Графического редактора прямоугольников в комплекте нет: разметку выполняют отдельно и передают таблицу. Координаты проверяются на выход за границы, пути — на выход из корня набора.

```
java -cp dist/ubor-sim.jar com.ubor.app.DatasetTool \
  crop ../data/raw ../data/boxes.csv ../data/crops
```

Для первоначального объезда заранее раскладывают безопасные предметы, выбирают понятный маршрут и избегают смешения нескольких задач в одном сеансе. Полезно отдельно записывать подъезд, небольшое боковое отклонение, остановку без предмета и контролируемый отказ. Не нужно собирать данные движением на опасные отходы: для

отказов достаточно безопасных макетов и обычных неподходящих предметов, не создающих угрозы механике.

Важна и отрицательная выборка: чистый пол, собственный бампер, полоска тени, край коврика. Набор из одних центрированных бутылок учит распознавать демонстрационную композицию, а не работать на машине. Для каждой записи сохраняют условия: покрытие, свет, расположение камеры, версию механики и настройки. После изменения высоты камеры нельзя автоматически объединять новую съёмку со старой без проверки совместимости.

18. Разбиение, обучение и оценка DL4J

Соседние кадры одного объезда очень похожи. Если случайно распределить их между обучением и тестом, модель фактически увидит почти те же сцены при проверке. Поэтому разделение проводится целыми сеансами. Все кадры одного session относятся только к train, val или test. Manifest проверяет это условие и отвергает повторное использование изображения.

Исходный рекордер намеренно пишет unassigned. Человек создаёт таблицу session,split. DatasetTool переносит выбранную принадлежность в новый манифест. Новый файл следует размещать рядом с деревом его изображений: загрузчик ML не допускает путей, выходящих за корень манифеста. Для объединённого датасета сначала организуют единый каталог изображений и сохраняют относительные пути без запятых.

```
java -cp dist/ubor-sim.jar com.ubor.app.DatasetTool \
  split ../data/raw/manifest.csv config/sessions.csv \
  ../data/raw/manifest.split.csv
```

Сеть в Networks — небольшой CNN: свёртка 5 × 5 с шагом 2 и 16 каналами, pooling, свёртка 3 × 3 с 32 каналами, глобальное усреднение, Dense 64 и выход. Для классификатора выход имеет шесть softmax-компонент и функцию потерь классификации; для модели движения — две tanh-компоненты и MSE. Это стартовая архитектура, а не результат сравнительного исследования нескольких моделей.

Trainer написан на Java, работает с мини-пакетами по восемь примеров, перемешивает только обучающую выборку, сохраняет лучший checkpoint по validation loss и затем отдельно оценивает test. Для классификатора требуется наличие всех классов в train. Для каждого выпуска записываются кривая обучения, метаданные, SHA-256, тестовая матрица ошибок либо MAE скорости и поворота.

```
# Требуются внешние зависимости и успешный native preflight
java -cp 'dist/full/*' com.ubor.ml.Trainer classifier \
  ../data/crops/manifest.csv ../models/classifier.zip 30

java -cp 'dist/full/*' com.ubor.ml.Trainer policy \
  ../data/raw/manifest.split.csv ../models/policy.zip 30
```

Число 30 здесь — пример числа эпох для запуска, а не найденный оптимум. Выбор checkpoint производится по валидации, но чрезмерное многократное использование одной валидационной выборки также приводит к подгонке эксперимента. Финальный тест нельзя использовать как ещё один набор для регулярной настройки порогов.

Для классификации важны не только общая accuracy и macro-F1, но и конкретные ошибки: сколько опасных макетов ошибочно признано бутылками, сколько бумаги пропущено на тёмном полу, сколько чистых окон объявлено мусором. В отчёте R01 отсутствующий класс в test помечается как неопределённый recall; это не ноль риска. Для модели движения MAE — вспомогательная метрика: небольшая средняя ошибка может скрывать один резкий опасный поворот.

Код обучения включён, но зависимости DL4J в этой среде не загружались, сети не обучались и реальных метрик нет. Любые числа точности до эксперимента были бы выдумкой. Файлы модели после обучения получают `approved.for.hardware=false`. Одобрение выполняется только после отдельной проверки изображений, локализации, временных задержек, соответствия классов и стендовых действий. Сам факт завершения Trainer не является разрешением двигаться.

19. Перенос ML на Raspberry Pi и проверка задержек

DL4J предоставляет Java-интерфейс, но вычисления ND4J используют нативный backend. Поэтому успешная компиляция Java на настольном компьютере ещё не доказывает загрузку численной библиотеки на Linux ARM64. В R01 закреплены DL4J/ND4J 1.0.0-M2.1 и JavaCV 1.5.7 с соответствующей линией OpenCV. Комбинация Java 21, выбранной ОС и ARM-зависимостей здесь не испытана; это самостоятельный этап допуска. [3, 16]

Полная сборка выполняется из software. Скрипт собирает все модули и копирует runtime-зависимости в `dist/full`. В отличие от компактного симулятора, этот каталог может быть большим из-за нативных платформенных библиотек. Для промышленного выпуска разумно оставить только нужные платформы после успешной проверки, но нельзя удалять артефакт, ориентируясь только на похожее название.

```
bash assemble-full.sh
java -cp 'dist/full/*' com.ubor.ml.Preflight
```

Preflight создаёт небольшой CNN и выполняет серию прямых проходов, выводя времена. Его цель — обнаружить проблему загрузки ND4J и получить начальное представление о задержке. Он не измеряет получение кадра, скользящие окна, нейросеть движения, запись диска, шину и реакцию контактора в сумме. Нельзя взять напечатанную медиану и назвать её задержкой всей машины.

Для полноценного временного испытания создают контролируемое визуальное событие и одновременно регистрируют соответствующий физический сигнал. Затем сопоставляют момент события, доступность кадра, окончание инференса, решение контроллера, запись PWM и остановку движения. Важно измерять хвост распределения, а не только среднее. Редкая длинная пауза GC или задержка камеры может быть важнее большинства быстрых кадров.

В текущем контроллере результат зрения старше 250 мс не используется для автоматического движения. При медленном скользящем окне машина будет останавливаться, что лучше скрытого движения по старой картинке, но может сделать концепцию практически бесполезной. Тогда сначала профилируют, уменьшают число кандидатов, меняют архитектуру или используют более подходящий детектор. Простое

увеличение допустимого возраста кадра без пересчёта остановки не является исправлением производительности.

Для CNN недостаточно оценить frames per second. При 0,20 м/с задержка 0,25 с соответствует 5 см движения ещё до исполнительного торможения. Если окно локализации ошибается на сопоставимую величину, губа может уже пройти нужную точку. Нужна совместная таблица ошибок: позиция объекта, угол подхода, освещение, возраст кадра, скорость и исход подбора.

Сервис systemd приведён как шаблон. Он запускается от отдельного пользователя, получает секрет из отдельного файла и имеет Restart=no, чтобы не восстанавливать бортовое приложение после отказа незаметно для оператора. Порт шаблона — 8844; примеры ручного запуска используют 9090, поэтому пульт должен подключаться к фактически выбранному порту. До установки сервиса аппаратный вариант запускают вручную на стенде и фиксируют версии всех компонентов.

20. Что показывают фактические проверки R01

В переданном комплекте выполнена настоящая офлайн-компиляция модулей core/app на Java 21.0.11. Собран JAR, запущены 50 именованных автоматических утверждений. Проверяются свежесть команд, отклонение NaN, диапазоны приводов, блокировки, фиксация отказов, ручной сброс, таймауты, переходы подбора, НМАС, защита от повторов, байтовые команды и отдельные свойства симулятора.

Проверка	Фактический результат	Граница результата
core/app javac и JAR	Выполнено успешно	Без Pi4J/DL4J
SelfTest	50 утверждений пройдены	Логика и модель, не физическая безопасность
TCP loopback	40 подписанных обменов	Не радиоканал на реальной площадке
Потеря сеанса	Отключение наблюдалось после паузы 300 мс	Не точное измерение времени остановки
Модельный подбор	3 предмета, 1 опасный оставлен	Oracle, не нейросеть
STEP round trip	121 валидное тело	Не FEA и не полный collision check
Электрический граф	141 компонент, 100 цепей	Не KiCad ERC, не SPICE и не плата
Swing GUI	Запуск и снимок экрана	Не полное тестирование действий пользователя

Сетевой тест использует настоящий сокет loopback, а не вызов метода контроллера вместо сети. Он отправляет 40 НМАС-команд, читает телеметрию и проверяет, что корректные команды разрешали модельные приводы. Затем соединение закрывается, и

через 300 мс проверяется запрет. Сам тест честно отмечает, что эта пауза наблюдения не определяет точную задержку выключения.

Модельный прогон длится 90 секунд симулированного времени. Его итог: `collected = 3`, `remaining = 1`, `last_reason = HAZARD_VISION_STOP`. Координаты и события записаны в `trajectory.csv`, карты — в PNG. Поле `mode` явно содержит `SIM_ORACLE_NOT_ML`, а поля `physical_hardware` и `electrical_circuit_simulation` равны `false`. Эти признаки нужны, чтобы отчёт нельзя было случайно принять за полевой эксперимент.

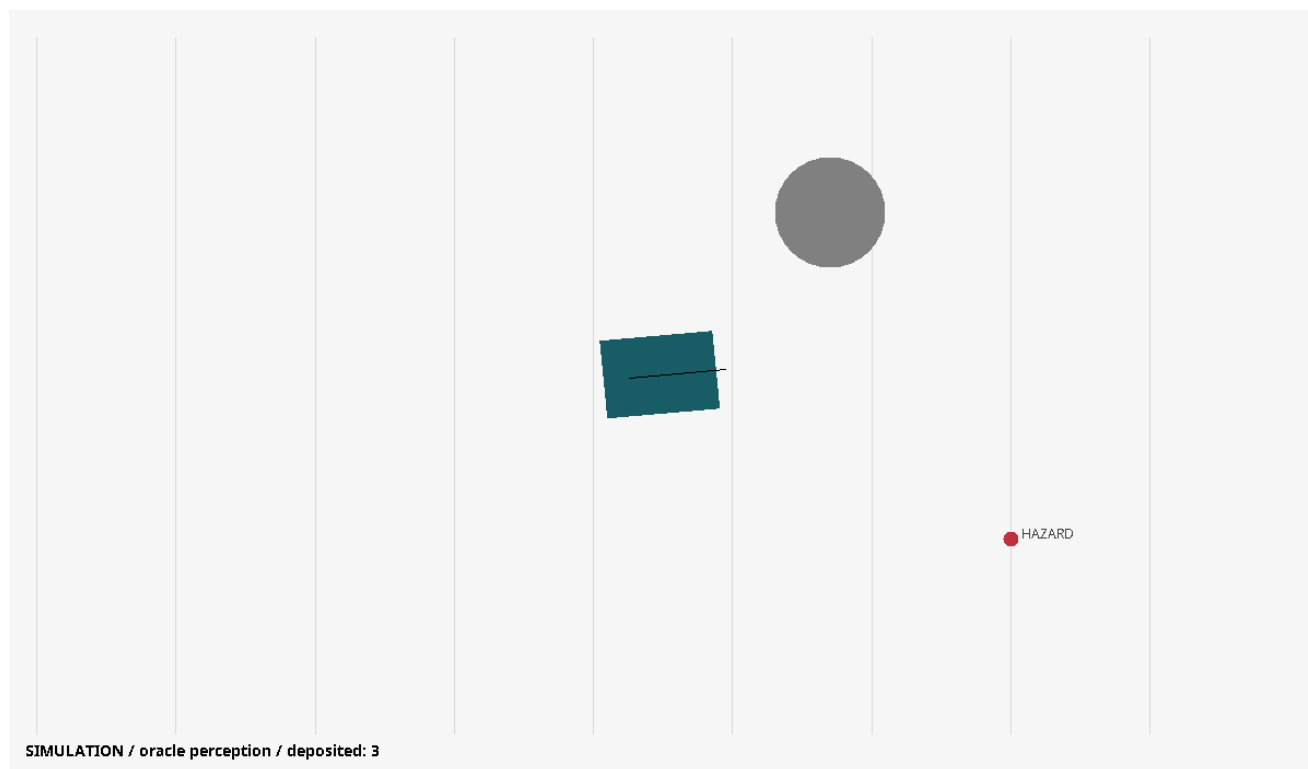


Рисунок 7. Итог модельного прогона: допускаемые предметы убраны, опасная цель осталась.

Для CAD выполнен повторный импорт STEP через геометрическое ядро и проверено число валидных тел. Для электроники проверены уникальность обозначений и номеров выводов, отсутствие моторной шины на GPIO, соответствие четырёх моторных каналов, соединение конденсатора VCP с VM и баланс синтаксических скобок KiCad-файлов. Не найдено цепей только с одним концом. Эти проверки полезны, но не понимают всех физических причин отказа.

Не выполнены загрузка и запуск KiCad, ERC, разводка PCB, SPICE, испытания GPIO, SPI на настоящем Pi, камера UVC, USB-геймпад, обучение DL4J, ARM native preflight и механические испытания. В книге и README это не спрятано за словом «готово». Готовыми здесь являются переданные исходники, CAD-геометрия, компактный исполняемый симулятор и перечисленные отчёты, а не физически испытанное изделие.

21. Стендовый ввод: последовательность разрешений

Стендовый ввод нельзя начинать с полностью собранной машины на полу и команды AUTO. Сначала должна появиться уверенность в каждом барьере, который удерживает систему от непреднамеренного движения. Работы проводят специалистом на ограждённом стенде, с доступным физическим отключением и безопасными макетами. Лабораторный источник с ограничением тока на ранних этапах предпочтительнее аккумулятора с большим доступным током.

Этап А — механическая и электрическая проверка без питания. Сверяют фактические детали с CAD, крепёж и стопорение валов, свободу вращения, защиту ремня и щётки, маршрут проводов. Проверяют полярность, отсутствие коротких замыканий, соответствие контактов разъёма и раздельность полюсов аварийной кнопки. Любая неподтверждённая соединительная колодка останавливает ввод, а не получает временную перемычку.

Этап В — логика без подключённых моторов. Проверяют 5,1 В и 3,3 В, начальный запрет RUN_EN, работу watchdog и ручного фронтального пуска. Отдельно имитируют удерживаемый ARM, исчезновение heartbeat, выключение питания, восстановление питания и открытый контакт кожуха. Разрешение не должно возвращаться автоматически после устранения причины. Сигнал K1_SENSE сверяют с настоящим состоянием контактора, а не только с напряжением катушки.

Этап С — силовая плата на ограниченном источнике. Проверяют форму PWM, направления, токовый сигнал и отключение nSLEEP. Измеряют бросок при предварительном заряде, отпуске контактора и напряжение VM при остановке. Пока не выбрано и испытано ограничение рекуперации, переход к энергичной нагрузке и батарее запрещён. Нагрев оценивается после выдержки режима, а не касанием платы через несколько секунд.

Этап D — вывешенные колёса, минимальные воздействия. Определяют знаки каждого двигателя и энкодера, измеряют число отсчётов на оборот колеса и проверяют нулевой выход при снятии разрешения. Здесь же нужна добавленная диагностика энкодерных отказов. Двигатели щётки и ленты ещё отключены. Неправильный знак обнаруживают до включения регулятора с существенным усилением.

Этап E — калибровка датчиков и остановки. Дальномер измеряют на нескольких известных расстояниях и материалах, выбирая монотонный рабочий диапазон. Проверяют контактные входы, лучи, датчик заполнения и край. Физический дальномер в компоновке расположен высоко; он не гарантирует обнаружение низких препятствий. Требуется согласовать его поле зрения с допустимым мусором и отдельно оценить слепую нижнюю зону. На R01 это дополнительное ограничение ограждённого полигона, а не разрешение игнорировать низкие препятствия.

Этап F — ручная езда на полу без подбора. Проверяют пульт, отпуске deadman, разрыв сети, выключение клиента, потерю фокуса и задержки. Затем отдельно запускают механизм приёма с безопасными предметами. В AUTO переходят только после проверки данных, моделей, калибровки камеры и полного временного тракта. На каждом

этапе сохраняют таблицу: версия, дата, условия, ожидаемое событие, измеренный результат и решение о переходе дальше.

22. Остановка и геометрический запас

Остановка — это сумма задержек и пути выбега. Если пропала команда, сначала истекает время её действия; затем выполняется решение или срабатывает аппаратный watchdog; затем снимается воздействие и механика замедляется. Электрическое отключение не равно мгновенной нулевой скорости. На склоне оно вообще не гарантирует удержание положения без отдельного тормоза.

Для предварительной оценки приняты $v = 0,30$ м/с, суммарная задержка 0,30 с, замедление $0,60$ м/с² и геометрический запас 0,10 м. Эти величины не измерены. Получается:

$$\begin{aligned}s &= v \cdot t + v^2 / (2 \cdot a) + \text{запас} \\s &= 0,30 \cdot 0,30 + 0,30^2 / (2 \cdot 0,60) + 0,10 \\s &\approx 0,265 \text{ м}\end{aligned}$$

В контроллере порог препятствия задан 0,35 м. Сопоставление 0,265 и 0,35 полезно только при одинаковой точке отсчёта. Если датчик расположен позади передней кромки щётки, его измеренное расстояние до препятствия нужно преобразовать к реальной выступающей части машины. В R01 такой геометрический вопрос должен быть разрешён измеренной калибровкой и расположением датчика, а не вычитанием произвольной величины из результата.

При полном бункере, другом покрытии, влажной поверхности или снижении сцепления фактический путь изменится. Если моторный драйвер в запрещённом состоянии даёт свободный выбег, расчёт с сильным активным торможением неверен. На стенде измеряют именно реализованный способ останова и его отказные варианты. Параметр deceleration логического симулятора не переносится в технический паспорт.

Отдельно проверяют остановку ленты и щётки. После выключения питания они сохраняют вращательную энергию, а гибкие лопасти могут продолжать двигаться. Размыкание кожуха должно запрещать дальнейшее приведение в движение, но безопасный доступ к механизму требует учёта остаточного движения. Для R01 режим обслуживания предполагает полное отключение, ожидание неподвижности и проверку остаточного напряжения.

У робота нет отдельного механического удерживающего тормоза и нет заявленной безопасной эксплуатации на уклоне после аварийного снятия питания. Поэтому расчёт 5° в главе 4 нельзя читать как гарантию остановки и удержания на пандусе. Если такая функция нужна, придётся добавить тормоз, датчик его состояния, управляющую схему и новые тесты, а также обновить CAD и массу.

23. Анализ отказов и незакрытые инженерные задачи

Разработка прототипа ускоряется, когда незавершённые вопросы имеют конкретные имена. «Нужно всё проверить» не помогает организовать работу; «не проверено напряжение VM при размыкании BMS» задаёт измеримый эксперимент. Таблица ниже является начальным анализом отказов, а не полной процедурой функциональной безопасности.

Отказ	Возможное последствие	Предусмотрено в R01 / что ещё нужно
Завис Java или шина	Сохранилась старая команда мотора	Heartbeat после I/O; внешний сброс; измерить полный останов
Потерялся пульт	Продолжение движения без оператора	Lease, закрытие сеанса, deadman; испытать реальный Wi-Fi
Оборван энкодер	Ошибка регулирования скорости	Счётчик есть; добавить монитор правдоподобия и отказные тесты
K1 приварился	Моторная шина осталась под напряжением	nSLEEP/OE — дополнительный запрет; оценить отказ одного барьера
BMS отключила батарею при торможении	Перенапряжение VM	Защита от рекуперации ещё не завершена; блокиер силового выпуска
Зажевало ленту	Нагрев, повреждение предмета	Ограничение тока, программная выдержка, timeout; стендовая настройка
Камера выдаёт старые кадры	Позднее решение	Проверка возраста Java-кадра; измерить буферизацию и экспозицию
Неподходящий объект получил высокий score	Неверный подбор	UNKNOWN/HAZARD, оператор; отдельная выборка отказов и ограничения
Переполнен бункер	Заклинивание зоны сброса	Вход заполнения; выбрать реальный датчик и геометрию
Сняли бункер при включённом механизме	Открытая зона падения/движения	Процедура обслуживания; дополнить блокировку наличия бункера

Таблица подчёркивает ещё одно ограничение: датчик заполнения не равен датчику присутствия бункера. В R01 отдельная проверенная блокировка наличия снятого бункера не реализована. Поэтому сервисная операция выполняется только при отключённом источнике; для автоматического безопасного обслуживания понадобится дополнительный датчик и изменение схемы и Java-входов.

До выпуска рабочих чертежей механики нужно выбрать моторы, редукторы, ступицы, муфты, подшипники и крепёж; уточнить посадки, натяжитель, стопорение валов и все

защитные кожухи. Условный объём покупного мотора не определяет расположение его крепёжных отверстий. Также нужен расчёт крепления батареи, опор бункера и кабелей с учётом вибрации.

До выпуска электроники нужны конкретные датчики, проект входных защит, выбор контактора и его полюсов, проверка предзаряда, защита от рекуперации, тепловой расчёт платы и разводка силовых петель. KiCad-файлы следует открыть в целевой версии, заменить условные интерфейсы фактическими символами и footprint, выполнить полноценный ERC, затем DRC платы и независимое ревью.

До аппаратного AUTO нужны полная Maven-компиляция, проверка native-библиотек на ARM64, калибровка, диагностика энкодеров, измеренная задержка и обученные модели. Нельзя закрыть эти пункты просто изменением `approved.for.hardware` или `calibration.complete`. Флаг — запись о выполненной процедуре, а не способ заменить процедуру.

R01 сознательно не включает SLAM, покрытие всей территории, объезд сложных препятствий, удалённую диспетчерскую, GPS-навигацию, автоматическую зарядку, уличную влагозащиту или работу среди людей. Разумная следующая ревизия начинается с одного выбранного расширения и измеримого критерия, а не с одновременного добавления всех возможностей.

24. Воспроизводимость и выпуск следующей ревизии

Воспроизводимый проект сохраняет не только исходники, но и условия их использования. Для R01 геометрические параметры находятся в `cad/parameters.json` и CAD-исходнике; электрические связи — в JSON, CSV и KiCad; интерфейсы программных данных — в Types; значения аппаратной калибровки — в `properties` и CSV. При изменении ширины подборщика нельзя забыть пороги в модели сбора и камере.

Повторная генерация CAD выполняется скриптом из `cad/src`. Он требует CadQuery и OpenCascade в подходящем окружении Python. Это инструмент проектирования, не часть бортового Java-runtime. Рендер строится из экспортированной сетки той же сцены. В комплект включены уже созданные STEP и PNG, поэтому открыть проект можно без установки генератора.

```
python cad/src/build_robot.py
python cad/src/render_cad.py
python tools/check_step.py
python tools/build_electronics.py
```

Генератор электроники создаёт схемы, таблицу соединений и отчёт структурных проверок из одного описания. Изменять только один PDF-лист вручную опасно: он разойдётся с каноническим графом. После существенной инженерной доработки в KiCad следует либо перенести изменение обратно в источник генерации, либо объявить новую CAD/ECAD-систему главным источником и больше не перезаписывать её старым скриптом.

Версию Java можно менять только вместе с проверкой библиотек и запуском тестов. Артефакт full следует строить в чистом каталоге, чтобы рядом не остались две версии

одной нативной зависимости. Датасет и модели имеют собственную версию: новый бинарный checkpoint не заменяет автоматически предыдущий, даже если его filename совпал.

Контрольные суммы в MANIFEST_SHA256.txt помогают убедиться, что файлы архива не повредились и что проверяется именно переданная ревизия. Они не являются цифровой подписью автора и не доказывают безопасность содержимого. Для каждого реального эксперимента полезно записывать hash JAR, hash моделей, конфигурацию, серийные номера датчиков, массу и состояние батареи.

Критерий завершения следующей ревизии лучше формулировать наблюдаемым результатом: «десять повторов потери пульта дают остановку в пределах измеренной дистанции на указанном покрытии при такой массе», а не «всё работает». Для сбора фиксируют долю успешных входов, долю подтверждённых выгрузок и причины отказов по классам и размерам. Для зрения сохраняют отдельную проверочную выборку. Для силовой части — осциллограммы и температуры.

Проект становится полезным не тогда, когда исчезают все слова «не проверено», а когда каждый такой пункт либо подтверждён экспериментом, либо сознательно исключён из области применения. R01 уже даёт связанный исходный материал для этой работы: CAD, электрический граф, Java-контур, пульт, модели обучения и воспроизводимый программный прогон.

Приложение А. Карта файлов и команды

Файл или каталог	Назначение
README_RU.md	Короткий вход в комплект и границы готовности
cad/step/*assembly.step	Основная твердотельная сборка
cad/step/*service.step	Снятый назад бункер
cad/parts/	Отдельные STEP-тела
cad/mechanical_bom.csv	Масса, материал и назначение элементов
cad/drawings/	SVG-проекции, DXF и эскизный PDF
electronics/kicad/	Корневой проект и 8 дочерних схем
electronics/wiring.csv	Соединения по выводам
electronics/connection_graph.json	Канонический электрический граф
software/dist/ubor-sim.jar	Проверенный исполняемый симулятор
software/config/	Закрытая аппаратная конфигурация, CSV-шаблоны
models/README_RU.md	Почему нет готовых весов
verification/	Отчёты, трасса, карты и снимок GUI

Сетевой smoke-тест запускается отдельно от SelfTest:

```
java -Djava.awt.headless=true -cp dist/ubor-sim.jar \  
com.ubor.app.NetworkSmoke ../verification/network-new.txt
```

Аппаратный AUTO после всех предусмотренных проверок:

```
java -cp 'dist/full/*' com.ubor.app.Main \  
--hardware config/robot.properties ../models 9090
```

Для локального симулятора не нужно задавать общий ключ: `--desktop` создаёт локальный секрет и связывает встроенный пульт с loopback-сервером. Для отдельных процессов `--server-sim`, `--controller` и аппаратного сервера требуется `UBOR_KEY`. Учебный raw-каталог должен быть новым: рекордер не перезаписывает существующий `manifest.csv`.

На Windows разделитель отдельных элементов classpath — точка с запятой, однако запись `-cp "dist/full/*"` остаётся одним элементом и раскрывается JVM. Полные аппаратные команды предназначены для Linux на Pi; Windows-пульт и симулятор не требуют доступа к GPIO.

Приложение В. Паспорт проверенной ревизии

Область	Статус на дату комплекта
Геометрия	121 тело, STEP повторно импортирован, топология валидна
Габарит STEP	809,5 × 545,0 × 665,0 мм
Масса	Оценка модели 24,31 кг; расчётная полная 30 кг
Бункер	Геометрический внутренний объём 24,12 л
Механический расчёт	Предварительные аналитические оценки, не FEA
Электроника	141 компонент, 100 цепей, 8 дочерних схем
KiCad	Исходники созданы; GUI/парсер KiCad и ERC не запускались
Программная сборка	core/app скомпилированы javac 21.0.11
Программные проверки	50 автоматических утверждений пройдены
Сеть	Реальный loopback TCP/НМАС, 40 обменов
Симуляция	90 модельных секунд; собрано 3; оставлен 1 HAZARD
GUI	Успешный запуск под Xvfb и снимок окна
Pi4J / DL4J	Исходники и POM есть; полный compile/run не выполнен
Данные и модели	Фото, обученные веса и реальные метрики отсутствуют
Производственный выпуск	Не выполнен; блокиеры перечислены в главе 23

Даты файлов и версия Java относятся к среде создания комплекта. Они не заменяют протокол повторного запуска на компьютере пользователя. Если любой тест после изменения кода не проходит, выпуск новой ревизии останавливается до выяснения причины; исправление теста ради зелёного результата не является исправлением машины.

Приложение С. Первичные источники и проверка предпосылок

Ниже приведены первичные документы производителей и проектов, использованные для выбора интерфейсов. Параметры конкретного изготовленного изделия, библиотеки и ОС необходимо повторно сверять перед закупкой и аппаратным выпуском. Числа расчётов и результаты собственных тестов берутся из файлов комплекта, а не из внешних публикаций.

[1] Pi4J. Release Notes: ветка 3, Java 21, выпуск 3.0.4 и исправление SPI.
<https://www.pi4j.com/about/release-notes/>

[2] Pi4J. LinuxFS и GpioD providers; технические ограничения и конфигурация.
<https://www.pi4j.com/documentation/providers/linuxfs/>
<https://www.pi4j.com/documentation/providers/gpiod/>

[3] Eclipse Deeplearning4j / Konduit. Основная документация DL4J и ND4J.
<https://deeplearning4j.konduit.ai/>

[4] Raspberry Pi. Raspberry Pi 5: питание и аппаратные сведения.
<https://www.raspberrypi.com/products/raspberry-pi-5/>

[5] Texas Instruments. DRV8874: режимы, токовая обратная связь, выводы и силовое включение.
<https://www.ti.com/lit/gpn/DRV8874>

[6] NXP. PCA9685: регистры, диапазон частоты, индивидуальные и общие записи PWM.
<https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf>

[7] Texas Instruments. TPS3431: внешний watchdog и настройка таймаута.
<https://www.ti.com/lit/gpn/TPS3431>

[8] Texas Instruments. TPS3431 evaluation module, руководство по включению.
<https://www.ti.com/lit/pdf/snvu614>

[9] Texas Instruments. SN74LVC1G74: фронтовый триггер, PRE и CLR.
<https://www.ti.com/lit/gpn/SN74LVC1G74>

[10] Texas Instruments. SN74AHCT1G125: логический буфер.
<https://www.ti.com/product/SN74AHCT1G125>

[11] onsemi. FQP30N06L: характеристики MOSFET, включая управление при 5 В.
<https://www.onsemi.com/pdf/datasheet/fqp30n06l-d.pdf>

[12] Microchip. MCP3008: АЦП и интерфейс SPI. <https://www.microchip.com/en-us/product/MCP3008>

[13] Microchip. MCP23S17: расширитель GPIO с SPI.
<https://www.microchip.com/en-us/product/MCP23S17>

[14] LSI Computer Systems. LS7366R: квадратурный счётчик и SPI-регистры.
<https://lsicsi.com/wp-content/uploads/2021/06/LS7366R.pdf>

[15] Linux kernel documentation. Joystick API / joydev.
<https://www.kernel.org/doc/html/latest/input/joydev/joystick-api.html>

[16] Deeplearning4j. Maven-конфигурация и платформенные зависимости.
<https://deeplearning4j.konduit.ai/configuration/maven>

В исходниках собственной программы использованы интерфейсы библиотек, а не копии их исходного кода. Зависимости сохраняют лицензии своих правообладателей. Этот комплект не распространяет шрифтовые файлы, каталожные модели сторонних производителей или чужие обученные веса.

25. WPILib в проекте UBOR

R02 заменяет способ расчёта движения, а не архитектуру управления. Пульт, сетевой протокол, SafetyController и DriveControl остаются теми же. Между сценой Simulator и физической моделью появился интерфейс SimDynamics; его реализация WpilibDynamics вызывает классы WPILib 2026.2.1.

Статус: исходный код интеграции подготовлен. Сборка и выполнение WPILib здесь НЕ проверены: Maven отсутствует, загрузка бинарных зависимостей недоступна. Работоспособность готового WPILib-дистрибутива этим выпуском не заявляется.

Как проходит один цикл

```
TCP/HMAC -> RobotService
-> SafetyController -> DriveControl -> Actuation
-> Simulator -> WpilibDynamics
-> физическое состояние и HAL-датчики
-> Sensors -> следующий цикл контроллера
```

Simulator больше не вычисляет координаты в WPILib-пути. Он получает готовую позу, скорости колёс, путь, скорости механизмов, токи и напряжение. Старый способ расчёта оставлен для регрессионной проверки и всегда называется LEGACY_FIRST_ORDER.

Что осталось от R01

Механика STEP/CAD и электрические схемы не менялись. Аппаратный адаптер Pi4J и код DL4J сохранены. Включение профиля wpilib не собирает профиль full и не требует загружать библиотеки обучения. В собственной программе нет кода C/C++; штатные нативные библиотеки WPILib используются через JNI.

Вариант запуска называется явно. run-simulator теперь вызывает WPILib; run-legacy-simulator вызывает старый JAR. Если WPILib не собран или не загружается JNI, запуск завершается ошибкой. Автоматического перехода к старой физике нет.

25.1. Приводы и батарея

Для дифференциального шасси применяется DifferentialDrivetrainSim. Щётка и приводной ролик транспортёра представлены двумя DCMotorSim. Напряжение на моторе получается из PWM-доли и напряжения шины, а не из прямого назначения скорости колесу.

Параметр	Значение / происхождение
Масса; колея; радиус	30 кг; 0,5 м; 0,1 м — исходные условия R01
Момент инерции по курсу	2,4 кг·м ² — оценка, не измерение CAD-массовой модели
Редукции: колёса / щётка / лента	112,5 / 25 / 38,8 — допущения
Моторы	Собственные наборы DCMotor на 24 В; не NEO/CIM из примера FRC
Батарея	25,6 В; 0,09 Ом; электроника 1 А — допущения
Радиус ролика	0,025 м; скорость ленты = угловая скорость × радиус

Конфигурация находится в config/wpilib-sim.properties. Параметры двигателя относятся к валу ДО редуктора; смещение паспортных характеристик редукторного выхода с коэффициентом редукции удвоит преобразование. Ширина колеи и радиус дополнительно сверяются с константами DriveControl.

Просадка питания

```
I_source = I_logic + sum(max(0, duty * I_winding))
V_loaded = BatterySim.calculateLoadedBatteryVoltage(
    V_open, R_internal, I_source)
```

Ток, который видит датчик мотора, — модуль тока обмотки. Для батареи используется оценка среднего входного тока PWM. Установлено напряжение моделируемой шины 25,6 В: стандартные 12 В из FRC-примеров не подставляются автоматически. Химия, заряд батареи, температура, токовые ограничения драйвера и рекуперативное перенапряжение здесь не моделируются.

25.2. HAL, время и отключение

WpiSensors записывает результат модели в EncoderSim, AnalogInputSim, DIOSim и RoboRioSim, затем считывает его через обычные объекты WPILib. Именно эти показания энкодеров и токов поступают в общий контроллер. Это проверяет программную обратную связь, но не исполняет SPI-протокол LS7366R/MCP3008 и не эмулирует Raspberry Pi.

Виртуальные каналы	Назначение
DIO 0–3	Два энкодера; метры и импульсы согласованы с DriveControl
AI 0	AnalogGyroSim: диагностический угол, не реальный IMU в схеме
AI 1 / AI 2–5	Дальность 1 В/м / токи 0,5 В/А; насыщение 0–5 В
DIO 4–12	NC-защиты, заполнение, разрешение, ошибка драйвера, два луча
RoboRioSim VIn	Симулируемая шина питания, а не питание физического roboRIO

Ось курса в физике положительна против часовой стрелки; AnalogGyro.getAngle использует противоположный знак. Адаптер учитывает это. Гироскоп используется для диагностики и отдельной проверки, а существующий контур DriveControl регулирует скорости по энкодерам.

Частоты и защёлки

Управляющий цикл работает с периодом 20 мс. Физика разбивается на подшаги до 5 мс, независимо от интерфейса пульта. Истечение 200-мс watchdog размыкает виртуальное разрешение; очистка аварийного признака не включает питание. Требуется новое действие Physical ARM. Пауза модели свыше 5 с отклоняется явно, а не маскируется потерей времени.

Обесточить моторы — не то же самое, что мгновенно остановить колёса. Нулевое входное напряжение в DC-модели соответствует электрическому торможению. Для разомкнутой цепи в R02 введена отдельная приближённая модель выбега с пассивным сопротивлением. Её коэффициент НЕ откалиброван; безопасный тормозной путь не подтверждён.

25.3. Сборка и работа с пультом

Требования: JDK 21, Maven 3.9+ и доступ к Maven Central/FRCMaven. В pom зафиксирована WPILib 2026.2.1. Схема сборки использует Java JAR и платформенные архивы *-cpp с JNI/shared-библиотеками. Maven распаковывает библиотеки в target/native; скрипты задают java.library.path и путь поиска зависимых DLL/SO.

```
cd software
mvn -Pwpilib clean verify
```

verify запускает реальные проверки WPILib отдельной JVM. Файлы Java из robot-pi/robot-mi не включаются, пока не выбран профиль full. Для Windows предусмотрен windowsx86-64, для Linux — linuxx86-64, для macOS — osxuniversal; эти платформенные сборки в данной среде не проверены. Linux ARM не является целевой конфигурацией этого настольного запуска.

Windows

```
run-wpilib.cmd --about
run-wpilib.cmd --desktop --nt
run-wpilib.cmd --demo ..\verification\my-demo 90
```

Linux

```
./run-wpilib.sh --about
./run-wpilib.sh --desktop --nt
./run-wpilib.sh --demo ../verification/my-demo 90
```

В пульте сначала нажать Physical ARM (SIM), затем Arm / Disarm и удерживать SPACE. Движение — стрелки/WASD; C — ручной подбор с человеческой меткой. Для записи добавить --record каталог. Это синтетические кадры симулятора, а не фотографии полигона.

Опция --nt публикует Field2d и телеметрию на 127.0.0.1:5810 для Glass. Команды управления по NetworkTables не принимаются. В headless-режиме --server доступны консольные команды arm, estop, restore, jam; сетевой сервер также по умолчанию слушает только loopback.

25.4. Что проверено, а что нет

Проверка в среде подготовки	Результат
Ядро + старый симулятор	50 проверок — PASS
Интерфейс сменной физики	20 проверок с TEST_DOUBLE_NOT_WPILIB — PASS
Конфигурация физических параметров	9 проверок без WPILib — PASS
TCP/HMAC на старой модели	40 обменов и останов после разрыва — PASS
Синтаксис Java	37 файлов, только parse — PASS
Компиляция / исполнение WPILib	НЕ ВЫПОЛНЕНО
WpiVerification	25 проверок подготовлены, НЕ выполнены

Двадцать проверок интерфейса подтверждают, что сцена берёт координаты, скорости и токи из подключённого backend; не пересчитывает их повторно; передаёт останов и заклинивание; закрывает ресурсы ровно один раз. Тестовая модель намеренно имеет отличное от WPILib имя и не подменяет классы edu.wpi.first.*.

Реальная suite WpiVerification должна проверить движение прямо и назад, поворот, знак гироскопа, энкодеры HAL, 25-вольтовую шину, выбег после отключения, вращение щётки/ролика, заклинивание и токи, watchdog, аварийные защёлки, полный сбор трёх предметов и TCP-отключение.

Где смотреть результат

```
robot-wpilib/target/wpilib-report/result.json
robot-wpilib/target/wpilib-report/wpilib-origin.txt
robot-wpilib/target/wpilib-report/oracle-demo/
verification/REPORT_R02_RU.md
```

В начале запуска статус заменяется на RUNNING_NOT_YET_PASSED. При исключении записывается FAIL; только завершение всей suite формирует PASS. Источник JAR фиксируется отдельно. Протоколы r01_archive и r02-core не являются свидетельством выполнения WPILib. Проверка parse не проверяет типы и внешние сигнатуры.

25.5. Границы модели и следующий стенд

Зрение в демонстрации называется SIM_ORACLE_NOT_ML: класс и координаты предмета известны сцене. WPILib не превращает такую демонстрацию в проверку DL4J. Качество детекции, обучение рулению, ложные срабатывания на тени и неизвестных предметах требуют отдельного набора данных и испытаний.

Сбор мусора остаётся геометрическим автоматом: предмет входит в зону подборщика, переходит на ленту и затем в бункер. Скорость продвижения теперь зависит от скорости ролика DCMotorSim. Если механизмы стоят, одной положительной команды недостаточно. Контакт не моделирует пластичность бутылки, изгиб бумаги, наматывание пакета и силы щетины.

Из сцены убрана ошибка R01: проверка collectible() в самой механике запрещала подбор опасного предмета даже при неверном управлении. Теперь материальный объект не имеет такой защиты, и запрет должен обеспечивать контроллер. Отдельная проверка интерфейса подтверждает возможность физического захвата объекта HAZARD при прямой неправильной команде.

Калибровка после появления прототипа

Сначала определить параметры купленных двигателей и редукторов, действительные импульсы энкодеров, массы и моменты инерции. На вывешенных колёсах снять скорость и ток как функции PWM/напряжения; на полу измерить разгон, поворот и свободный выбег с разной загрузкой. Сравнивать временные ряды энкодеров и токов, а не только конечную точку маршрута.

Для этапа сборщика отдельно измерить разгон щётки и ленты, ток при контакте с эталонными предметами и последствия наматывания. Сначала завершить независимую аварийную цепь и физические испытания защиты. Программная симуляция не обосновывает безопасное использование рядом с людьми.

Первичные источники

WPILib: Physics Simulation, Drivetrain Simulation Tutorial, Device Simulation, API DCMotorSim/BatterySim. Исходные сигнатуры и состав зависимостей сверены с репозиторием [wpilibsuite/allwpilib](https://github.com/wpilibsuite/allwpilib), тег v2026.2.1. Полные адреса приведены в [docs/WPILIB_SOURCES.md](#). Новая глава дополняет R01 и не снимает его инженерных ограничений.